

llmsverifier_integration_content_specs

LLMsVerifier Integration Plan — Detailed Content Specifications

HelixTranslate (digital.vasic.translator) +
LLMsVerifier (digital.vasic.llmsverifier)

PART A: FOUNDATION INTEGRATION

Chapter 1: Module & Dependency Management

1.1 Section Title & Purpose

Title: "Module Integration: Adding the LLMsVerifier Dependency" **Purpose:** Document the exact changes to go.mod to import the LLMsVerifier module with proper require/replace directives, including version constraints and transitive dependency resolution.

1.2 Specific Content Points

Content Point 1.2.1: Document the current go.mod structure

- Show the current go.mod header: `module digital.vasic.translator, go 1.25.2`
- List all existing require directives (19 direct dependencies)
- List existing replace directives: `digital.vasic.challenges => ./Challenges, digital.vasic.containers => ./Containers`

Content Point 1.2.2: Define the new require directive

- Exact line to add: `digital.vasic.llmsverifier v0.0.1`

- Explain semantic version pinning strategy
- Document transitive dependencies that LLMsVerifier brings:
 - github.com/spf13/viper v1.19.0 (config loading)
 - github.com/mattn/go-sqlite3 v1.14.24 (database — same version, verify compatible)
 - github.com/sashabaranov/go-openai v1.37.0 (OpenAI client library)
 - Others to be enumerated after `go mod tidy`

Content Point 1.2.3: Define the replace directive

- Exact line to add: `digital.vasic.llmsverifier => ./LLMsVerifier`
- Explain that LLMsVerifier must be cloned/cloned as submodule at repo root
- Document Git submodule setup alternative:

```
[submodule "LLMsVerifier"]
    path = LLMsVerifier
    url = git@github.com:vasic-digital/LLMsVerifier.git
```

Content Point 1.2.4: Document the go mod tidy execution sequence

- Step-by-step commands:
 1. `cd /path/to/HelixTranslate`
 2. `echo "replace digital.vasic.llmsverifier => ./LLMsVerifier" >> go.mod`
 3. `echo "require digital.vasic.llmsverifier v0.0.0" >> go.mod`
 4. `go mod tidy`
 5. `go mod verify`

Content Point 1.2.5: Document dependency conflict resolution strategy

- File: `go.mod` — manual conflict resolution for version mismatches
- Common conflicts to anticipate:
 - github.com/spf13/viper: HelixTranslate may not use viper directly; LLMsVerifier needs v1.19.0
 - github.com/mattn/go-sqlite3: Both use v1.14.24 — should be compatible
 - gopkg.in/yaml.v3: HelixTranslate has v3.0.1; LLMsVerifier also uses v3

1.3 Required Tables

Table 1.3.1: go.mod Directive Changes | Directive | Current State | New State | File

	-----	-----	-----		module	<code>digital.vasic.translator</code>	
unchanged		<code>go.mod</code>		require (new)		—	<code>digital.vasic.llmsverifier v0.0.1</code>
	<code>go.mod</code>		replace (new)		—	<code>digital.vasic.llmsverifier => ./</code>	
LLMsVerifier		<code>go.mod</code>		require viper		absent	<code>github.com/spf13/viper</code>
v1.19.0 (transitive)		<code>go.mod</code>	(auto)				

Table 1.3.2: Transitive Dependencies from LLMsVerifier | Dependency | Version

Purpose	Conflict Risk		-----	-----	-----		<code>github.com/</code>
<code>spf13/viper</code>		v1.19.0		YAML/JSON/TOML config loading		Low	<code>github.com/</code>
<code>sashabaranov/go-openai</code>		v1.37.0		OpenAI API client		Low	<code>github.com/</code>
<code>mitchellh/mapstructure</code>		v1.5.0		Struct mapping		Low	<code>github.com/fsnotify/</code>
<code>fsnotify</code>		v1.7.0		File watch for config reload		Low	

1.4 Required Code Blocks

Code Block 1.4.1: Final go.mod require section (partial)

```
require (
    // ... existing 19 dependencies ...
    digital.vasic.llmsverifier v0.0.1
)

replace (
    digital.vasic.challenges => ./Challenges
    digital.vasic.containers => ./Containers
    digital.vasic.llmsverifier => ./LLMsVerifier
)
```

Code Block 1.4.2: Verification commands

```
# Verify module resolution
go list -m digital.vasic.llmsverifier
# Expected output: digital.vasic.llmsverifier v0.0.1

# Verify build
go build ./...

# Run a simple import test
cat > /tmp/verify_import.go << 'EOF'
package main
import _ "digital.vasic.llmsverifier/llm-verifier/config"
func main() {}
EOF
go run /tmp/verify_import.go
```

1.5 Configuration Examples

- .gitmodules addition for LLMsVerifier submodule
- CI/CD pipeline modification to clone LLMsVerifier before go mod tidy

1.6 Test Specifications

- **Unit Test:** go.mod parse test — verify module directive exists
- **Integration Test:** go build ./... passes with zero errors after module addition
- **Test File:** test/integration/module_resolution_test.go
- **Test Function:** TestModuleResolution — imports each LLMsVerifier subpackage and verifies no build errors

1.7 Documentation References

- LLMsVerifier go.mod at LLMsVerifier/go.mod — reference for version requirements
 - HelixTranslate CLAUDE.md section on module structure
-

Chapter 2: Configuration System Integration

2.1 Section Title & Purpose

Title: "Configuration Integration: LLMsVerifierConfig Struct & Environment Variables" **Purpose:** Document the addition of the LLMsVerifier configuration section to HelixTranslate's config system, including the new struct definition, 16+ environment variables, and the configs/verifier.yaml file.

2.2 Specific Content Points

Content Point 2.2.1: Define LLMsVerifierConfig struct

- **File to modify:** internal/config/config.go
- **Location:** Add new field to root Config struct
- **Exact signature:**

```
// Config (existing) – add field:
type Config struct {
    // ... existing fields ...
    LLMsVerifier LLMsVerifierConfig
    `json:"llmsverifier,omitempty"`
    mapstructure:"llmsverifier"
}
```

Content Point 2.2.2: Define LLMsVerifierConfig struct (complete)

- **File to create/modify:** internal/config/config.go (append)
- **Exact struct definition:**

```
// LLMsVerifierConfig holds all LLMsVerifier integration settings
type LLMsVerifierConfig struct {
    Enabled bool `json:"enabled" mapstructure:"enabled"`
    BaseURL string `json:"base_url" mapstructure:"base_url"`
    APIKey string `json:"api_key" mapstructure:"api_key"`
    DatabasePath string `json:"database_path" mapstructure:"database_path"`
    VerificationEnabled bool `json:"verification_enabled" mapstructure:"verification_enabled"`
    ScoringEnabled bool `json:"scoring_enabled" mapstructure:"scoring_enabled"`
    DiscoveryEnabled bool `json:"discovery_enabled" mapstructure:"discovery_enabled"`
    AutoVerifyOnStartup bool `json:"auto_verify_on_startup" mapstructure:"auto_verify_on_startup"`
    AutoScoreOnStartup bool `json:"auto_score_on_startup" mapstructure:"auto_score_on_startup"`
}
```

```

MaxConcurrentTests    int
    `json:"max_concurrent_tests"
    mapstructure:"max_concurrent_tests"`
VerificationTimeout   time.Duration
    `json:"verification_timeout"
    mapstructure:"verification_timeout"`
ScoringWeights        ScoringWeightsConfig
    `json:"scoring_weights" mapstructure:"scoring_weights"`
Providers              []VerifierProviderConfig
    `json:"providers" mapstructure:"providers"`
EventIntegration       EventIntegrationConfig
    `json:"event_integration"
    mapstructure:"event_integration"`
APIIntegration         APIIntegrationConfig
    `json:"api_integration" mapstructure:"api_integration"`
}

```

Content Point 2.2.3: Define sub-structs

- **File:** internal/config/config.go

```

// ScoringWeightsConfig defines weight overrides for scoring components
type ScoringWeightsConfig struct {
    ResponseSpeed    float64 `json:"response_speed"
        mapstructure:"response_speed"`
    ModelEfficiency  float64 `json:"model_efficiency"
        mapstructure:"model_efficiency"`
    CostEffectiveness float64 `json:"cost_effectiveness"
        mapstructure:"cost_effectiveness"`
    Capability        float64 `json:"capability"
        mapstructure:"capability"`
    Recency           float64 `json:"recency"
        mapstructure:"recency"`
}

// VerifierProviderConfig defines a single provider to verify
type VerifierProviderConfig struct {
    Name      string `json:"name" mapstructure:"name"`
    Endpoint  string `json:"endpoint"
        mapstructure:"endpoint"`
    APIKey    string `json:"api_key"
        mapstructure:"api_key"`
    Model     string `json:"model,omitempty"
        mapstructure:"model"`
    Headers   map[string]string `json:"headers,omitempty"
        mapstructure:"headers"`
}

// EventIntegrationConfig controls event system integration
type EventIntegrationConfig struct {
    Enabled      bool `json:"enabled"
        mapstructure:"enabled"`
    SubscribeTopics []string `json:"subscribe_topics"
        mapstructure:"subscribe_topics"`
}

```

```

        PublishResults    bool    `json:"publish_results"
        mapstructure:"publish_results"`
    }

    // APIIntegrationConfig controls API endpoint registration
    type APIIntegrationConfig struct {
        Enabled            bool    `json:"enabled" mapstructure:"enabled"`
        EnableVerify        bool    `json:"enable_verify"
        mapstructure:"enable_verify"`
        EnableModels        bool    `json:"enable_models"
        mapstructure:"enable_models"`
        EnableScore          bool    `json:"enable_score"
        mapstructure:"enable_score"`
    }

```

Content Point 2.2.4: Define environment variable loading

- **File to modify:** internal/config/config.go — add to existing env var loading function
- **16+ environment variables:** | Variable | Purpose | maps to |
|-----|-----|-----| | LMSVERIFIER_ENABLED | Master enable switch |
LLMsVerifierConfig.Enabled | | LMSVERIFIER_BASE_URL | LLMsVerifier
API base URL | .BaseURL | | LMSVERIFIER_API_KEY | API authentication key
| .APIKey | | LMSVERIFIER_DB_PATH | SQLite database path | .DatabasePath
| | LMSVERIFIER_VERIFICATION_ENABLED | Enable verification pipeline |
.VerificationEnabled | | LMSVERIFIER_SCORING_ENABLED | Enable
scoring system | .ScoringEnabled | | LMSVERIFIER_DISCOVERY_ENABLED |
Enable model discovery | .DiscoveryEnabled | |
LMSVERIFIER_AUTO_VERIFY | Run verification on startup |
.AutoVerifyOnStartup | | LMSVERIFIER_AUTO_SCORE | Run scoring on
startup | .AutoScoreOnStartup | | LMSVERIFIER_MAX_CONCURRENT | Max
concurrent verification tests | .MaxConcurrentTests | |
LMSVERIFIER_TIMEOUT | Verification timeout (duration string) |
.VerificationTimeout | | LMSVERIFIER_WEIGHT_SPEED | Scoring weight:
response speed | .ScoringWeights.ResponseSpeed | |
LMSVERIFIER_WEIGHT_EFFICIENCY | Scoring weight: model efficiency |
.ScoringWeights.ModelEfficiency | | LMSVERIFIER_WEIGHT_COST |
Scoring weight: cost effectiveness | .ScoringWeights.CostEffectiveness | |
LMSVERIFIER_WEIGHT_CAPABILITY | Scoring weight: capability |
.ScoringWeights.Capability | | LMSVERIFIER_WEIGHT_RECENCY | Scoring
weight: recency | .ScoringWeights.Recency |

Content Point 2.2.5: Add default configuration function

- **File:** internal/config/config.go

```

// DefaultLLMsVerifierConfig returns the default LLMsVerifier
configuration
func DefaultLLMsVerifierConfig() LLMsVerifierConfig {
    return LLMsVerifierConfig{
        Enabled:            false,
        BaseURL:            "http://localhost:8081",
        DatabasePath:        "./data/llmsverifier.db",
        VerificationEnabled: true,
        ScoringEnabled:      true,
    }
}

```

```

    DiscoveryEnabled:    true,
    AutoVerifyOnStartup: false,
    AutoScoreOnStartup: false,
    MaxConcurrentTests: 5,
    VerificationTimeout: 30 * time.Second,
    ScoringWeights: ScoringWeightsConfig{
        ResponseSpeed:    0.25,
        ModelEfficiency:  0.20,
        CostEffectiveness: 0.25,
        Capability:        0.20,
        Recency:           0.10,
    },
    EventIntegration: EventIntegrationConfig{
        Enabled:          true,
        SubscribeTopics: []string{"translation.completed",
"translation.started"},
        PublishResults:   true,
    },
    APIIntegration: APIIntegrationConfig{
        Enabled:          true,
        EnableVerify:     true,
        EnableModels:     true,
        EnableScore:      true,
    },
}
}

```

Content Point 2.2.6: Create configs/verifier.yaml

- **File to create:** configs/verifier.yaml
- **Complete YAML content:**

```

llmsverifier:
  enabled: false
  base_url: "http://localhost:8081"
  api_key: "${LLMSVERIFIER_API_KEY}"
  database_path: "./data/llmsverifier.db"
  verification_enabled: true
  scoring_enabled: true
  discovery_enabled: true
  auto_verify_on_startup: false
  auto_score_on_startup: false
  max_concurrent_tests: 5
  verification_timeout: "30s"
  scoring_weights:
    response_speed: 0.25
    model_efficiency: 0.20
    cost_effectiveness: 0.25
    capability: 0.20
    recency: 0.10
  providers:
    - name: "openai"
      endpoint: "https://api.openai.com/v1"

```

```

    api_key: "${OPENAI_API_KEY}"
    model: "gpt-4"
  - name: "anthropic"
    endpoint: "https://api.anthropic.com/v1"
    api_key: "${ANTHROPIC_API_KEY}"
    model: "claude-3-sonnet"
  - name: "deepseek"
    endpoint: "https://api.deepseek.com/v1"
    api_key: "${DEEPSEEK_API_KEY}"
    model: "deepseek-chat"
event_integration:
  enabled: true
  subscribe_topics:
    - "translation.completed"
    - "translation.started"
  publish_results: true
api_integration:
  enabled: true
  enable_verify: true
  enable_models: true
  enable_score: true

```

Content Point 2.2.7: Add config loading from YAML

- **File to modify:** internal/config/config.go
- Add function LoadVerifierConfig(path string) (*LLMsVerifierConfig, error) that uses viper to load the YAML file and overlay environment variables.

2.3 Required Tables

Table 2.3.1: Config Struct Changes | Struct | Field Added | Type | Tags | Default |

Struct	Field Added	Type	Tags	Default
Config	LLMsVerifier			
LLMsVerifierConfig	json:"llmsverifier,omitempty"			zero value

Table 2.3.2: Environment Variable Mapping | Env Var | Config Path | Type |

Env Var	Config Path	Type
Default	Required	
LLMSVERIFIER_ENABLED	llmsverifier.enabled	bool false No
LLMSVERIFIER_BASE_URL	llmsverifier.base_url	string "http://localhost:8081" No
LLMSVERIFIER_API_KEY	llmsverifier.api_key	string "" No
LLMSVERIFIER_DB_PATH	llmsverifier.database_path	string "./data/llmsverifier.db" No
LLMSVERIFIER_VERIFICATION_ENABLED	llmsverifier.verification_enabled	bool true No
LLMSVERIFIER_SCORING_ENABLED	llmsverifier.scoring_enabled	bool true No
LLMSVERIFIER_DISCOVERY_ENABLED	llmsverifier.discovery_enabled	bool true No
LLMSVERIFIER_AUTO_VERIFY	llmsverifier.auto_verify_on_startup	bool false No
LLMSVERIFIER_AUTO_SCORE	llmsverifier.auto_score_on_startup	bool false No
LLMSVERIFIER_MAX_CONCURRENT	llmsverifier.max_concurrent_tests	int 5 No
LLMSVERIFIER_TIMEOUT	llmsverifier.verification_timeout	duration 30s No

2.4 Required Code Blocks

Code Block 2.4.1: Config struct modification in internal/config/config.go

```
type Config struct {
    Server      ServerConfig      `json:"server"
    mapstructure:"server"`
    Security    SecurityConfig    `json:"security"
    mapstructure:"security"`
    Translation TranslationConfig `json:"translation"
    mapstructure:"translation"`
    Preparation PreparationConfig `json:"preparation"
    mapstructure:"preparation"`
    Distributed DistributedConfig `json:"distributed"
    mapstructure:"distributed"`
    Logging     LoggingConfig     `json:"logging"
    mapstructure:"logging"`
    LLMsVerifier LLMsVerifierConfig
    `json:"llmsverifier,omitempty"
    mapstructure:"llmsverifier"`
}
```

Code Block 2.4.2: Viper-based config loading

```
func LoadVerifierConfig(path string) (*LLMsVerifierConfig, error)
{
    v := viper.New()
    v.SetConfigFile(path)
    v.SetEnvPrefix("LLMSVERIFIER")
    v.AutomaticEnv()

    if err := v.ReadInConfig(); err != nil {
        return nil, fmt.Errorf("failed to read verifier config:
        %w", err)
    }

    var cfg LLMsVerifierConfig
    if err := v.Unmarshal(&cfg); err != nil {
        return nil, fmt.Errorf("failed to unmarshal verifier
        config: %w", err)
    }

    return &cfg, nil
}
```

2.5 Configuration Examples

- Complete configs/verifier.yaml (see 2.2.6 above)
- .env.example additions for 13+ API keys (see 2.2.4 for env vars)
- JSON config snippet for config.json integration

2.6 Test Specifications

- **Test File:** internal/config/config_test.go (new) or test/unit/config_verifier_test.go
- **Test Functions:**
 - TestDefaultLLMsVerifierConfig — verify all defaults match specification
 - TestLoadVerifierConfigFromYAML — load from configs/verifier.yaml, verify all fields
 - TestLoadVerifierConfigEnvOverride — set env vars, verify override works
 - TestLLMsVerifierConfigValidation — validate invalid configs return errors
- **Table-driven tests** for each environment variable mapping

2.7 Documentation References

- LLMsVerifier: llm-verifier/config/config.go — reference struct patterns
 - HelixTranslate: internal/config/config.go — existing config loading patterns
-

Chapter 3: Provider Expansion (9 to 30+)

3.1 Section Title & Purpose

Title: "Provider Expansion: LLMsVerifier Adapter for 30+ LLM Providers"
Purpose: Document the creation of the LLMsVerifier provider adapter that bridges HelixTranslate's LLMClient interface with LLMsVerifier's 30+ provider backends, expanding from 9 to 30+ supported providers.

3.2 Specific Content Points

Content Point 3.2.1: List all 30+ providers from LLMsVerifier Current HelixTranslate providers (9): openai, anthropic, deepseek, zhipu, qwen, gemini, ollama, llamacpp, mock

New providers added via LLMsVerifier adapter (21+):

#	Provider	Source File	Auth Env Var
1	openai	already exists	OPENAI_API_KEY
2	anthropic	already exists	ANTHROPIC_API_KEY
3	deepseek	already exists	DEEPSEEK_API_KEY
4	zhipu	already exists	ZHIPU_API_KEY
5	qwen	already exists	QWEN_API_KEY
6	gemini	already exists	GEMINI_API_KEY
7	ollama	already exists	N/A
8	llamacpp	already exists	N/A
9	mock	already exists	N/A
10	cohere	providers/cohere.go	COHERE_API_KEY
11	groq	providers/groq.go	GR0Q_API_KEY
12	xai	providers/xai.go	XAI_API_KEY
13	togetherai	providers/togetherai.go	TOGETHERAI_API_KEY
14	replicate	providers/replicate.go	REPLICATE_API_TOKEN
15	cloudflare	providers/cloudflare.go	CLOUDFLARE_API_KEY
16	hyperbolic	providers/hyperbolic.go	HYPERBOLIC_API_KEY
17	sambanova	providers/sambanova.go	SAMBANOVA_API_KEY
18	siliconflow	providers/	

siliconflow.go | SILICONFLOW_API_KEY || 19 | **upstage** | providers/upstage.go | UPSTAGE_API_KEY || 20 | **publicai** | providers/publicai.go | PUBLICAI_API_KEY || 21 | **kimi** | providers/kimi.go | KIMI_API_KEY || 22 | **kimicode** | providers/kimicode.go | KIMICODE_API_KEY || 23 | **cerebras** | providers/cerebras.go | CEREBRAS_API_KEY || 24 | **modal** | providers/modal.go | MODAL_API_KEY || 25 | **nia** | providers/nia.go | NIA_API_KEY || 26 | **mistral** | providers/mistral.go | MISTRAL_API_KEY || 27 | **vulavula** | providers/vulavula.go | VULAVULA_API_KEY || 28 | **novita** | providers/novita.go | NOVITA_API_KEY || 29 | **sarvam** | providers/sarvam.go | SARVAM_API_KEY || 30 | **kilo** | providers/kilo.go | KILO_API_KEY || 31 | **nlpccloud** | providers/nlpccloud.go | NLPCLLOUD_API_KEY |

Content Point 3.2.2: Define the adapter pattern

- **File to create:** internal/providers/llmsverifier/adapter.go
- **Pattern:** Adapter — wraps LLMsVerifier's provider service to implement HelixTranslate's LLMClient interface
- **Exact interface implementation:**

```
// LLMsVerifierAdapter wraps LLMsVerifier providers to implement
// HelixTranslate's LLMClient
type LLMsVerifierAdapter struct {
    providerName string
    modelName    string
    client       *llmsverifier.Client // from
                digital.vasic.llmsverifier/sdk/go
    config       *ProviderConfig
}

// Compile-time interface check
var _ llm.LLMClient = (*LLMsVerifierAdapter)(nil)

func (a *LLMsVerifierAdapter) Translate(ctx context.Context, text
    string, prompt string) (string, error) {
    // Delegate to LLMsVerifier client's completion API
}

func (a *LLMsVerifierAdapter) GetProviderName() string {
    return fmt.Sprintf("llmsverifier-%s", a.providerName)
}
```

Content Point 3.2.3: Modify pkg/translator/llm/llm.go

- Add new Provider constant: ProviderLLMsVerifier Provider = "llmsverifier"
- Add to ValidModels map: entries for each LLMsVerifier-supported model
- Add factory case in NewLLMTranslator:

```
case ProviderLLMsVerifier:
    return NewLLMsVerifierAdapter(config)
```

Content Point 3.2.4: Create internal/providers/llmsverifier/provider.go

- Full provider implementation file

Table 3.3.2: Required Environment Variables (21 new)					Variable	Provider	Status
					COHERE_API_KEY	cohere	New
					GROQ_API_KEY	groq	New
					XAI_API_KEY	xai	New
					TOGETHERAI_API_KEY	togetherai	New
					REPLICATE_API_TOKEN	replicate	New
					CLOUDFLARE_API_KEY	cloudflare	New
					HYPERBOLIC_API_KEY	hyperbolic	New
					SAMBANOVA_API_KEY	sambanova	New
					SILICONFLOW_API_KEY	siliconflow	New
					UPSTAGE_API_KEY	upstage	New
					PUBLICAI_API_KEY	publicai	New
					KIMI_API_KEY	kimi	New
					KIMICODE_API_KEY	kimicode	New
					CEREBRAS_API_KEY	cerebras	New
					MODAL_API_KEY	modal	New
					NIA_API_KEY	nia	New
					MISTRAL_API_KEY	mistral	New
					VULAVULA_API_KEY	vulavula	New
					NOVITA_API_KEY	novita	New
					SARVAM_API_KEY	sarvam	New
					KILO_API_KEY	kilo	New

3.4 Required Code Blocks

Code Block 3.4.1: LLMsVerifierAdapter implementation skeleton

```
// internal/providers/llmsverifier/adapters.go
package llmsverifier

import (
    "context"
    "fmt"
    "time"

    llmverifier "digital.vasic.llmsverifier/llm-verifier/sdk/go"
    llmverifierconfig "digital.vasic.llmsverifier/llm-verifier/config"
    "digital.vasic.translator/pkg/translator/llm"
)

// LLMsVerifierAdapter wraps LLMsVerifier SDK to implement
// HelixTranslate LLMClient
type LLMsVerifierAdapter struct {
    providerName string
    modelName    string
    client       *llmverifier.Client
    config       *ProviderConfig
}

// Compile-time interface verification
var _ llm.LLMClient = (*LLMsVerifierAdapter)(nil)

// NewLLMsVerifierAdapter creates a new adapter from
// HelixTranslate provider config
func NewLLMsVerifierAdapter(config llm.ProviderConfig)
    (*LLMsVerifierAdapter, error) {
    lvConfig := llmverifierconfig.LLMConfig{
        Name:      config.Provider,
        Endpoint:  config.BaseURL,
        APIKey:    config.APIKey,
        Model:     config.Model,
    }
}
```

```

    client, err := llmverifier.NewClient(lvConfig)
    if err != nil {
        return nil, fmt.Errorf("failed to create LLMsVerifier
            client: %w", err)
    }

    return &LLMsVerifierAdapter{
        providerName: config.Provider,
        modelName:    config.Model,
        client:       client,
    }, nil
}

// Translate implements llm.LLMClient
func (a *LLMsVerifierAdapter) Translate(ctx context.Context, text
    string, prompt string) (string, error) {
    if a.client == nil {
        return "", fmt.Errorf("LLMsVerifier client not
            initialized")
    }

    fullPrompt := fmt.Sprintf("%s\n\n%s", prompt, text)

    resp, err := a.client.CreateCompletion(ctx,
        llmverifier.CompletionRequest{
            Model:      a.modelName,
            Prompt:   fullPrompt,
            MaxTokens: 4096,
            Temperature: 0.3,
        })
    if err != nil {
        return "", fmt.Errorf("LLMsVerifier completion failed:
            %w", err)
    }

    return resp.Text, nil
}

// GetProviderName implements llm.LLMClient
func (a *LLMsVerifierAdapter) GetProviderName() string {
    return fmt.Sprintf("llmsverifier-%s", a.providerName)
}

```

Code Block 3.4.2: Factory modification in pkg/translator/llm/llm.go

```

// Provider enum addition
const (
    // ... existing providers ...
    ProviderLLMsVerifier Provider = "llmsverifier"
)

// ValidModels addition
var ValidModels = map[Provider][]string{

```

```

// ... existing entries ...
ProviderLLMsVerifier: {
    "auto", // Auto-detect from LLMsVerifier
},
}

// Factory switch addition
func NewLLMTranslator(config ProviderConfig) (*LLMTranslator,
    error) {
    switch config.Provider {
// ... existing cases ...
    case ProviderLLMsVerifier:
        adapter, err :=
            llmsverifier.NewLLMsVerifierAdapter(config)
        if err != nil {
            return nil, fmt.Errorf("failed to create LLMsVerifier
                adapter: %w", err)
        }
        return &LLMTranslator{client: adapter}, nil
// ...
    }
}

```

3.5 Configuration Examples

- configs/verifier.yaml provider list (see Chapter 2)
- .env.example with all 21 new API key entries
- Per-provider config JSON examples

3.6 Test Specifications

- **Test File:** internal/providers/llmsverifier/adapter_test.go
- **Test Functions:**
 - TestLLMsVerifierAdapter_ImplementsInterface — compile-time interface check
 - TestLLMsVerifierAdapter_Translate — mock client, verify translation flow
 - TestLLMsVerifierAdapter_GetProviderName — verify naming convention
 - TestLLMsVerifierAdapter_ProviderSwitch — table-driven test for all 30+ providers
 - TestNewLLMsVerifierAdapter_InvalidConfig — error cases
- **Mock:** Create mock LLMsVerifier client for unit testing

3.7 Documentation References

- LLMsVerifier: llm-verifier/providers/ directory — 30+ provider implementations
 - LLMsVerifier: llm-verifier/sdk/go/client.go — SDK client interface
 - HelixTranslate: pkg/translator/llm/llm.go — existing provider factory
-

Chapter 4: Verification Pipeline Integration (8-Step)

4.1 Section Title & Purpose

Title: "Verification Pipeline: 8-Step Model Verification at Translation Startup"

Purpose: Document the integration of LLMsVerifier's 8-step verification pipeline that runs at translation startup, verifying model availability, capabilities, and quality before translation begins.

4.2 Specific Content Points

Content Point 4.2.1: Define the 8-step verification pipeline Based on internal/verifier/verification.go reference (8-test verification):

Step	Name	Purpose	Reference Function
1	Model Existence Check	Verify model ID exists in registry	VerifyModelExists()
2	API Connectivity Test	Ping provider API endpoint	VerifyAPIConnectivity()
3	Authentication Validation	Verify API key is valid	VerifyAuthentication()
4	Basic Completion Test	Send simple prompt, verify response	VerifyBasicCompletion()
5	Capability Detection	Query supported features	VerifyCapabilities()
6	Translation Quality Test	Send translation task, score output	VerifyTranslationQuality()
7	Latency Benchmark	Measure response time percentiles	VerifyLatencyBenchmarks()
8	Error Handling Verification	Verify graceful error responses	VerifyErrorHandling()

Content Point 4.2.2: Create internal/verifier/pipeline.go

- **File to create:** internal/verifier/pipeline.go
- Define VerificationPipeline struct and Step interface:

```
package verifier

import (
    "context"
    "fmt"
    "time"

    llmverifier "digital.vasic.llmsverifier/llm-verifier/verification"
    llmverifierdb "digital.vasic.llmsverifier/database"
)

// VerificationPipeline orchestrates the 8-step verification process
type VerificationPipeline struct {
    verifier    *llmverifier.Verifier
    db          *llmverifierdb.Database
    config      *VerificationConfig
    steps       []VerificationStep
    results     *PipelineResult
}
```



```

// VerificationStep represents a single verification step
type VerificationStep interface {
    Name() string
    Execute(ctx context.Context, state *StepState) (*StepResult,
        error)
    Timeout() time.Duration
    IsCritical() bool // If true, pipeline fails on error
}

// StepState holds accumulated state across steps
type StepState struct {
    ModelID      string
    ProviderName string
    APIKey       string
    BaseURL      string
    Results      map[string]*StepResult
}

// PipelineResult aggregates all step results
type PipelineResult struct {
    StepsCompleted int           `json:"steps_completed"`
    StepsFailed    int           `json:"steps_failed"`
    TotalSteps     int           `json:"total_steps"`
    DurationMs     int64        `json:"duration_ms"`
    StepResults    []*StepResult `json:"step_results"`
    Passed         bool          `json:"passed"`
}

// StepResult holds a single step's outcome
type StepResult struct {
    StepName  string `json:"step_name"`
    Passed    bool   `json:"passed"`
    DurationMs int64  `json:"duration_ms"`
    Details   map[string]any `json:"details,omitempty"`
    Error     string `json:"error,omitempty"`
}

```

Content Point 4.2.3: Implement each of the 8 steps as separate types

- **File:** internal/verifier/steps.go (or individual files)

```

// Step 1: Model Existence Check
type ModelExistenceStep struct{}
func (s *ModelExistenceStep) Name() string { return "model_existence" }
func (s *ModelExistenceStep) Timeout() time.Duration { return 5 * time.Second }
func (s *ModelExistenceStep) IsCritical() bool { return true }
func (s *ModelExistenceStep) Execute(ctx context.Context, state *StepState) (*StepResult, error) {
    // Check model exists in LLMsVerifier registry
}

// Step 2: API Connectivity Test

```

```

type APIConnectivityStep struct{}
func (s *APIConnectivityStep) Name() string { return
    "api_connectivity" }
func (s *APIConnectivityStep) Timeout() time.Duration { return 10
    * time.Second }
func (s *APIConnectivityStep) IsCritical() bool { return true }
func (s *APIConnectivityStep) Execute(ctx context.Context, state
    *StepState) (*StepResult, error) {
    // HTTP HEAD/GET to provider endpoint
}

// Steps 3-8 follow same pattern...

```

Content Point 4.2.4: Create pipeline orchestrator

- File: internal/verifier/orchestrator.go

```

// RunPipeline executes the full 8-step pipeline for a given model
func (p *VerificationPipeline) RunPipeline(ctx context.Context,
    modelID string) (*PipelineResult, error) {
    start := time.Now()
    state := &StepState{
        ModelID: modelID,
        Results: make(map[string]*StepResult),
    }

    result := &PipelineResult{
        TotalSteps: len(p.steps),
        StepResults: make([]*StepResult, 0, len(p.steps)),
    }

    for _, step := range p.steps {
        stepCtx, cancel := context.WithTimeout(ctx,
            step.Timeout())
        stepResult, err := step.Execute(stepCtx, state)
        cancel()

        if err != nil {
            stepResult = &StepResult{
                StepName: step.Name(),
                Passed: false,
                Error: err.Error(),
            }
            result.StepsFailed++
            if step.IsCritical() {
                result.Passed = false
                result.DurationMs =
                    time.Since(start).Milliseconds()
                return result, fmt.Errorf("critical step %q
                    failed: %w", step.Name(), err)
            }
        } else {
            result.StepsCompleted++
            state.Results[step.Name()] = stepResult
        }
    }
}

```

```

    }

    result.StepResults = append(result.StepResults,
                                stepResult)
}

result.Passed = result.StepsFailed == 0
result.DurationMs = time.Since(start).Milliseconds()
return result, nil
}

```

Content Point 4.2.5: Integration with translation startup

- **File to modify:** cmd/unified-translator/main.go
- Add verification call before translation:

```

func executeTranslation(session *TranslationSession) error {
    // NEW: Run verification pipeline if LLMsVerifier is enabled
    if session.Config.LLMsVerifier.Enabled &&
        session.Config.LLMsVerifier.AutoVerifyOnStartup {
        pipeline :=
            verifier.NewVerificationPipeline(session.VerifierConfig)
        result, err := pipeline.RunPipeline(ctx,
            session.Config.Provider.Model)
        if err != nil || !result.Passed {
            return fmt.Errorf("verification pipeline failed: %w
                (passed=%v, steps=%d/%d)",
                    err, result.Passed, result.StepsCompleted,
                    result.TotalSteps)
        }
        log.Printf("Verification pipeline passed: %d/%d steps in
            %dms",
                result.StepsCompleted, result.TotalSteps,
                result.DurationMs)
    }

    // Continue with existing translation flow...
}

```

4.3 Required Tables

Table 4.3.1: 8-Step Verification Pipeline									
Step #	Name	Function Name	Timeout	Critical	Metric				
1	Model Existence	ModelExistenceStep	5s	Yes	model_found bool				API Connectivity
2	Authentication	APIConnectivityStep	10s	Yes	latency_ms				AuthenticationStep
3	Basic Completion	auth_valid bool	10s	Yes					BasicCompletionStep
4	Capability Detection	response_quality	30s	Yes					CapabilityDetectionStep
5	Translation Quality	capabilities[]	15s	No					TranslationQualityStep
6	Latency Benchmark	quality_score	60s	No					LatencyBenchmarkStep
7	Error Handling	p50/p95/p99	120s	No					ErrorHandlingStep
8		error_graceful bool	30s	No					

Table 4.3.2: Pipeline Result Structure	Field	Type	Description
	StepsCompleted	int	Count of passed steps
	StepsFailed	int	Count of failed steps
	TotalSteps	int	Always 8
	DurationMs	int64	Total pipeline duration
	StepResults	[]StepResult	Per-step details
	Passed	bool	True if all critical steps passed

4.4 Required Code Blocks

Code Block 4.4.1: Complete VerificationPipeline struct and constructor

```
// internal/verifier/pipeline.go
package verifier

import (
    "context"
    "fmt"
    "time"

    llmverifier "digital.vasic.llmsverifier/llm-verifier/
        verification"
)

// NewVerificationPipeline creates a pipeline with default 8 steps
func NewVerificationPipeline(config *VerificationConfig)
    *VerificationPipeline {
    return &VerificationPipeline{
        config: config,
        steps: []VerificationStep{
            &ModelExistenceStep{},
            &APIConnectivityStep{},
            &AuthenticationStep{},
            &BasicCompletionStep{},
            &CapabilityDetectionStep{},
            &TranslationQualityStep{},
            &LatencyBenchmarkStep{},
            &ErrorHandlingStep{},
        },
    }
}

// VerificationConfig holds pipeline configuration
type VerificationConfig struct {
    Enabled          bool
    TimeoutPerStep  time.Duration
    MaxConcurrent   int
    SkipSteps       []string
}
```

Code Block 4.4.2: Integration hook in translation startup

```
// internal/verifier/startup_hook.go
package verifier
```

```

import (
    "context"
    "fmt"
    "log"
)

// VerifyBeforeTranslation runs the 8-step pipeline before
// allowing translation
func VerifyBeforeTranslation(ctx context.Context, cfg
    *VerificationConfig, modelID string) error {
    if !cfg.Enabled {
        log.Println("LLMsVerifier pipeline disabled, skipping
            verification")
        return nil
    }

    pipeline := NewVerificationPipeline(cfg)
    result, err := pipeline.RunPipeline(ctx, modelID)

    if err != nil {
        return fmt.Errorf("verification pipeline error: %w", err)
    }

    if !result.Passed {
        return fmt.Errorf("verification pipeline failed: %d/%d
            steps passed",
            result.StepsCompleted, result.TotalSteps)
    }

    log.Printf("Verification complete: %d/%d steps passed in
        %dms",
        result.StepsCompleted, result.TotalSteps,
        result.DurationMs)
    return nil
}

```

4.5 Configuration Examples

```

# configs/verifier.yaml – verification section
llmsverifier:
  verification:
    enabled: true
    timeout_per_step: 30s
    max_concurrent: 5
    skip_steps: [] # Optional: ["latency_benchmark",
        "error_handling"]
    quality_threshold: 7.0 # Minimum quality score (0-10)

```

4.6 Test Specifications

- **Test File:** internal/verifier/pipeline_test.go
- **Test Functions:**
 - TestVerificationPipeline_AllStepsPass — mock all steps passing

- `TestVerificationPipeline_CriticalStepFails` — simulate critical failure
- `TestVerificationPipeline_NonCriticalStepFails` — simulate non-critical failure, verify continuation
- `TestVerificationPipeline_Timeout` — verify timeout handling per step
- `TestVerificationPipeline_SkipSteps` — verify skip configuration works
- `TestVerifyBeforeTranslation_Disabled` — verify no-op when disabled
- **Integration Test:** `test/integration/verification_pipeline_test.go`

4.7 Documentation References

- `LLMsVerifier`: `llm-verifier/verification/verification.go` — verification types
 - `HelixTranslate`: `cmd/unified-translator/main.go` — startup flow
-

Chapter 5: Scoring Integration (5-Component)

5.1 Section Title & Purpose

Title: "Scoring Integration: 5-Component Model Ranking for Translation Quality"

Purpose: Document the integration of `LLMsVerifier`'s 5-component scoring system that ranks models for translation tasks based on speed, efficiency, cost, capability, and recency.

5.2 Specific Content Points

Content Point 5.2.1: Reference the 5 scoring components from `LLMsVerifier` Based on `llm-verifier/scoring/types.go` and `scoring_engine.go`:

Component	Weight (default)	Description	Data Source
Response Speed	0.25	Token throughput, latency	Benchmark measurements
Model Efficiency	0.20	Parameter utilization, context efficiency	models.dev API
Cost Effectiveness	0.25	\$/token, \$/quality unit	Provider pricing
Capability	0.20	Feature support, translation-specific	Capability registry
Recency	0.10	Release date, update frequency	models.dev API

Content Point 5.2.2: Create `internal/verifier/scoring.go`

- **File to create:** `internal/verifier/scoring.go`

```
package verifier
```

```
import (
    "context"
    "fmt"
    "math"
    "time"
```

```

    llmverifierscoring "digital.vasic.llmsverifier/llm-verifier/
        scoring"
    llmverifierdb "digital.vasic.llmsverifier/database"
)

// ScoringIntegrator wraps LLMsVerifier scoring for HelixTranslate
type ScoringIntegrator struct {
    engine *llmverifierscoring.ScoringEngine
    config *ScoringConfig
}

// ScoringConfig holds scoring configuration for translation
type ScoringConfig struct {
    Enabled          bool          `json:"enabled"`
    Weights          ScoreWeights `json:"weights"`
    MinAcceptableScore float64      `json:"min_acceptable_score"`
    AutoScoreOnStartup bool        `json:"auto_score_on_startup"`
    ScoreCacheTTL    time.Duration `json:"score_cache_ttl"`
}

// ScoreWeights defines component weights (must sum to 1.0)
type ScoreWeights struct {
    ResponseSpeed    float64 `json:"response_speed"`
    ModelEfficiency  float64 `json:"model_efficiency"`
    CostEffectiveness float64 `json:"cost_effectiveness"`
    Capability       float64 `json:"capability"`
    Recency          float64 `json:"recency"`
}

// TranslationModelScore extends LLMsVerifier score with
// translation-specific metrics
type TranslationModelScore struct {
    llmverifierscoring.ComprehensiveScore
    TranslationSpecific struct {
        TranslationQualityScore float64 `json:"translation_quality_score"`
        SourceLanguageSupport   []string `json:"source_language_support"`
        TargetLanguageSupport   []string `json:"target_language_support"`
        FormatSupport            []string `json:"format_support"`
        CulturalAdaptationScore float64 `json:"cultural_adaptation_score"`
    } `json:"translation_specific"`
}

```

Content Point 5.2.3: Implement scoring functions

- CalculateScore(ctx, modelID string) (*TranslationModelScore, error)

- RankModelsForTranslation(ctx, sourceLang, targetLang string) ([]TranslationModelScore, error)
- GetBestModelForTranslation(ctx, sourceLang, targetLang string, minScore float64) (*TranslationModelScore, error)
- IsModelAcceptable(score *TranslationModelScore, threshold float64) bool

Content Point 5.2.4: Create score adapter service

- **File:** internal/services/llmsverifier_score_adapter.go
- Bridges scoring results to translation engine model selection

package services

```
import (
    "context"
    "fmt"
    "sort"
```

```
    "digital.vasic.translator/internal/verifier"
```

```
)
```

```
// ScoreAdapter bridges LLMsVerifier scoring to HelixTranslate model selection
```

```
type ScoreAdapter struct {
    scoring *verifier.ScoringIntegrator
}
```

```
// SelectBestModel returns the highest-scoring model for a translation task
```

```
func (sa *ScoreAdapter) SelectBestModel(ctx context.Context,
    candidates []string, sourceLang, targetLang string) (
    string, float64, error) {
    var scoredModels []struct {
        modelID string
        score    float64
    }

    for _, modelID := range candidates {
        score, err := sa.scoring.CalculateScore(ctx, modelID)
        if err != nil {
            continue // Skip unscorable models
        }
        scoredModels = append(scoredModels, struct {
            modelID string
            score    float64
        }{modelID, score.OverallScore})
    }

    if len(scoredModels) == 0 {
        return "", 0, fmt.Errorf("no models could be scored")
    }
```

```
// Sort by score descending
```



```

    sort.Slice(scoredModels, func(i, j int) bool {
        return scoredModels[i].score > scoredModels[j].score
    })

    return scoredModels[0].modelID, scoredModels[0].score, nil
}

```

5.3 Required Tables

Table 5.3.1: 5-Component Score Breakdown | Component | Weight | Data Sources | Calculation Method | Range | |-----|-----|-----|-----|-----| |

Response Speed	0.25	Latency benchmarks, throughput	Normalized inverse latency	0-10	
Model Efficiency	0.20	Parameter count, context window	params/context ratio	0-10	
Cost Effectiveness	0.25	Input/output \$/1M tokens	Cost-performance ratio	0-10	
Capability	0.20	Feature matrix, translation tests	Weighted capability score	0-10	
Recency	0.10	Release date	Time-decay function	0-10	

Table 5.3.2: Score Interpretation for Translation | Overall Score | Rating | Recommendation | |-----|-----|-----| |

9.0-10.0	Excellent	Primary choice for critical translations
7.5-8.9	Very Good	Recommended for production use
6.0-7.4	Good	Suitable with monitoring
4.0-5.9	Fair	Use with caution, verify output
0.0-3.9	Poor	Not recommended for translation

5.4 Required Code Blocks

Code Block 5.4.1: ScoringIntegrator complete implementation

```

// internal/verifier/scoring.go
package verifier

// NewScoringIntegrator creates a scoring integrator with given
// config
func NewScoringIntegrator(engine
    *llmverifierscoring.ScoringEngine, config *ScoringConfig)
    *ScoringIntegrator {
    return &ScoringIntegrator{
        engine: engine,
        config: config,
    }
}

// CalculateScore computes a translation-specific comprehensive
// score
func (si *ScoringIntegrator) CalculateScore(ctx context.Context,
    modelID string) (*TranslationModelScore, error) {
    // Fetch base score from LLMsVerifier engine
    baseScore, err := si.engine.CalculateComprehensiveScore(ctx,
        modelID, llmverifierscoring.DefaultScoringConfig())
    if err != nil {
        return nil, fmt.Errorf("failed to calculate base score:
            %w", err)
    }
}

```

```

// Build translation-specific score
txScore := &TranslationModelScore{
    ComprehensiveScore: *baseScore,
}

// Add translation-specific metrics
txScore.TranslationSpecific.TranslationQualityScore =
    si.calculateTranslationQuality(ctx, modelID)
txScore.TranslationSpecific.SourceLanguageSupport =
    si.getSupportedSourceLanguages(modelID)
txScore.TranslationSpecific.TargetLanguageSupport =
    si.getSupportedTargetLanguages(modelID)
txScore.TranslationSpecific.FormatSupport = []string{"fb2",
    "epub", "txt", "html", "pdf", "docx"}
txScore.TranslationSpecific.CulturalAdaptationScore = 7.5 //
    Default, enhanced with verification

return txScore, nil
}

// IsModelAcceptable checks if a model meets the minimum quality
// threshold
func (si *ScoringIntegrator) IsModelAcceptable(score
    *TranslationModelScore, threshold float64) bool {
    if threshold == 0 {
        threshold = si.config.MinAcceptableScore
    }
    return score.OverallScore >= threshold
}

```

Code Block 5.4.2: Score caching mechanism

```

// internal/verifier/scoring_cache.go
package verifier

import (
    "sync"
    "time"
)

// ScoreCache provides TTL-based caching for model scores
type ScoreCache struct {
    mu      sync.RWMutex
    entries map[string]*cachedScore
    ttl     time.Duration
}

type cachedScore struct {
    score      *TranslationModelScore
    cachedAt  time.Time
}

```

```

func (sc *ScoreCache) Get(modelID string)
    (*TranslationModelScore, bool) {
    sc.mu.RLock()
    defer sc.mu.RUnlock()
    entry, ok := sc.entries[modelID]
    if !ok || time.Since(entry.cachedAt) > sc.ttl {
        return nil, false
    }
    return entry.score, true
}

func (sc *ScoreCache) Set(modelID string, score
    *TranslationModelScore) {
    sc.mu.Lock()
    defer sc.mu.Unlock()
    sc.entries[modelID] = &cachedScore{
        score:    score,
        cachedAt: time.Now(),
    }
}

```

5.5 Configuration Examples

```

# configs/verifier.yaml – scoring section
llmsverifier:
  scoring:
    enabled: true
    min_acceptable_score: 6.0
    auto_score_on_startup: false
    score_cache_ttl: "1h"
    weights:
      response_speed: 0.25
      model_efficiency: 0.20
      cost_effectiveness: 0.25
      capability: 0.20
      recency: 0.10

```

5.6 Test Specifications

- **Test File:** internal/verifier/scoring_test.go
- **Test Functions:**
 - TestScoringIntegrator_CalculateScore — mock engine, verify score calculation
 - TestScoringIntegrator_IsModelAcceptable — table-driven tests for thresholds
 - TestScoreCache_GetSet — verify TTL caching behavior
 - TestScoreCache_Expiration — verify expired entries are refreshed
 - TestScoreAdapter_SelectBestModel — verify ranking and selection logic
- **Integration Test:** test/integration/scoring_integration_test.go

5.7 Documentation References

- LLMsVerifier: `llm-verifier/scoring/scoring_engine.go` — scoring calculation
 - LLMsVerifier: `llm-verifier/scoring/types.go` — score type definitions
-

Chapter 6: Model Discovery Integration (3-Tier)

6.1 Section Title & Purpose

Title: "Model Discovery: 3-Tier Discovery System for Translation Models"

Purpose: Document the 3-tier model discovery system that finds, registers, and verifies available models across 30+ providers for translation tasks.

6.2 Specific Content Points

Content Point 6.2.1: Define the 3-tier discovery architecture | Tier | Name | Scope | Function | Refresh | |-----|-----|-----|-----|-----| | Tier 1 | **Static Registry** | Built-in known models | `LoadStaticRegistry()` | Per release | | Tier 2 | **Provider Discovery** | Query each provider's API | `DiscoverFromProviders()` | On startup | | Tier 3 | **Dynamic Detection** | Live capability probing | `DynamicCapabilityDetection()` | On demand |

Content Point 6.2.2: Tier 1 — Static Registry

- **File to create:** `internal/verifier/discovery_static.go`
- Embedded list of known translation-capable models per provider

```
// StaticRegistry contains known-good models for translation
var StaticRegistry = map[string][]StaticModelEntry{
    "openai": {
        {ModelID: "gpt-4", Name: "GPT-4", ContextWindow: 8192,
         TranslationQuality: 9.5},
        {ModelID: "gpt-4-turbo", Name: "GPT-4 Turbo",
         ContextWindow: 128000, TranslationQuality: 9.5},
        {ModelID: "gpt-4o", Name: "GPT-4o", ContextWindow:
         128000, TranslationQuality: 9.3},
        {ModelID: "gpt-3.5-turbo", Name: "GPT-3.5 Turbo",
         ContextWindow: 16385, TranslationQuality: 8.0},
    },
    "anthropic": {
        {ModelID: "claude-3-opus", Name: "Claude 3 Opus",
         ContextWindow: 200000, TranslationQuality: 9.6},
        {ModelID: "claude-3-sonnet", Name: "Claude 3 Sonnet",
         ContextWindow: 200000, TranslationQuality: 9.2},
        {ModelID: "claude-3-haiku", Name: "Claude 3 Haiku",
         ContextWindow: 200000, TranslationQuality: 8.5},
    },
    // ... entries for all 30+ providers
}
```

```

type StaticModelEntry struct {
    ModelID      string `json:"model_id"`
    Name         string `json:"name"`
    ContextWindow int    `json:"context_window"`
    TranslationQuality float64 `json:"translation_quality"`
    Languages    []string `json:"languages"`
}

```

Content Point 6.2.3: Tier 2 — Provider Discovery

- **File to create:** internal/verifier/discovery_provider.go

// ProviderDiscovery queries each provider's API for available models

```

type ProviderDiscovery struct {
    clients map[string]ProviderAPIClient
    config  *DiscoveryConfig
}

```

// DiscoveryResult holds discovered models from a provider

```

type DiscoveryResult struct {
    Provider string `json:"provider"`
    Models   []DiscoveredModel `json:"models"`
    Error    string `json:"error,omitempty"`
    DurationMs int64 `json:"duration_ms"`
}

```

// DiscoveredModel represents a model found via provider API

```

type DiscoveredModel struct {
    ModelID      string `json:"model_id"`
    Name         string `json:"name"`
    Created      int64 `json:"created,omitempty"`
    ContextWindow int    `json:"context_window"`
    Pricing      ModelPricing `json:"pricing,omitempty"`
    Capabilities map[string]bool `json:"capabilities"`
}

```

// DiscoverAll queries all configured providers

```

func (pd *ProviderDiscovery) DiscoverAll(ctx context.Context)
    ([]DiscoveryResult, error) {
    var results []DiscoveryResult
    var mu sync.Mutex
    var wg sync.WaitGroup

    for providerName, client := range pd.clients {
        wg.Add(1)
        go func(name string, c ProviderAPIClient) {
            defer wg.Done()
            result := pd.discoverProvider(ctx, name, c)
            mu.Lock()
            results = append(results, result)
            mu.Unlock()
        }(providerName, client)
    }
}

```

```

    }

    wg.Wait()
    return results, nil
}

```

Content Point 6.2.4: Tier 3 — Dynamic Detection

- **File to create:** internal/verifier/discovery_dynamic.go
- Live capability probing for unknown models

```

// DynamicDetector performs live tests to detect model
// capabilities
type DynamicDetector struct {
    verifier *VerificationPipeline
}

// DetectionResult holds dynamically detected capabilities
type DetectionResult struct {
    ModelID          string          `json:"model_id"`
    Provider         string          `json:"provider"`
    Capabilities     []string        `json:"capabilities"`
    TranslationQuality float64         `json:"translation_quality"`
    TestResults      []TestResult    `json:"test_results"`
}

// DetectCapabilities runs a series of tests to discover model
// capabilities
func (dd *DynamicDetector) DetectCapabilities(ctx
    context.Context, modelID, provider string)
    (*DetectionResult, error) {
    tests := []CapabilityTest{
        &BasicCompletionTest{},
        &TranslationTest{SourceLang: "en", TargetLang: "sr"},
        &LongContextTest{MinTokens: 16000},
        &JSONModeTest{},
        &StreamingTest{},
    }

    result := &DetectionResult{
        ModelID: modelID,
        Provider: provider,
    }

    for _, test := range tests {
        testResult, err := test.Run(ctx, modelID, provider)
        if err == nil && testResult.Passed {
            result.Capabilities = append(result.Capabilities,
                test.CapabilityName())
        }
        result.TestResults = append(result.TestResults,
            *testResult)
    }
}

```

```

    return result, nil
}

```

Content Point 6.2.5: Discovery aggregator

- **File to create:** internal/verifier/discovery.go
- Combines all 3 tiers into unified model catalog

```

// DiscoveryOrchestrator manages the 3-tier discovery process
type DiscoveryOrchestrator struct {
    staticRegistry *StaticRegistry
    providerDiscovery *ProviderDiscovery
    dynamicDetector *DynamicDetector
    config          *DiscoveryConfig
}

// RunFullDiscovery executes all 3 tiers and merges results
func (do *DiscoveryOrchestrator) RunFullDiscovery(ctx
    context.Context) (*UnifiedModelCatalog, error) {
    catalog := &UnifiedModelCatalog{
        GeneratedAt: time.Now(),
        Models:      make(map[string]CatalogEntry),
    }

    // Tier 1: Static registry (always available)
    for provider, models := range do.staticRegistry.GetAll() {
        for _, model := range models {
            catalog.Models[model.ModelID] = CatalogEntry{
                ModelID:      model.ModelID,
                Provider:      provider,
                Name:           model.Name,
                SourceTier:     "static",
                Capabilities: []string{"translation"},
            }
        }
    }

    // Tier 2: Provider discovery
    if do.config.EnableProviderDiscovery {
        results, _ := do.providerDiscovery.DiscoverAll(ctx)
        for _, result := range results {
            for _, model := range result.Models {
                if entry, exists :=
                    catalog.Models[model.ModelID]; exists {
                    entry.SourceTier = "provider+static"
                    entry.LastDiscovered = time.Now()
                } else {
                    catalog.Models[model.ModelID] = CatalogEntry{
                        ModelID:      model.ModelID,
                        Provider:      result.Provider,
                        Name:           model.Name,
                        SourceTier:     "provider",

```

```

        Capabilities: model.CapabilitiesToList(),
    }
}
}
}

// Tier 3: Dynamic detection (on-demand for selected models)
if do.config.EnableDynamicDetection {
    for modelID, entry := range catalog.Models {
        if entry.NeedsDynamicDetection() {
            detection, _ :=
do.dynamicDetector.DetectCapabilities(ctx, modelID,
entry.Provider)
            entry.Capabilities = detection.Capabilities
            entry.TranslationQuality =
detection.TranslationQuality
            catalog.Models[modelID] = entry
        }
    }
}

return catalog, nil
}

```

6.3 Required Tables

Table 6.3.1: 3-Tier Discovery Summary

Tier	Source	Latency	Coverage
1	Static	Compiled registry	Accuracy 0ms Known models only High (pre-verified)
2	Provider	Provider APIs	5-30s All advertised models Medium (may include deprecated)
3	Dynamic	Live testing	30-300s Tested subset Highest (verified)

Table 6.3.2: Discovery Configuration

Setting	Default	Range	Description
enable_provider_discovery	true	bool	Enable Tier 2 discovery
enable_dynamic_detection	false	bool	Enable Tier 3 (slow)
provider_timeout	10s	5-60s	Timeout per provider API call
dynamic_test_count	3	1-10	Number of tests for Tier 3
cache_ttl	1h	0-24h	How long to cache discovery results

6.4 Required Code Blocks

Code Block 6.4.1: Complete discovery orchestrator

```

// internal/verifier/discovery.go
package verifier

import (
    "context"
    "fmt"
    "sync"
    "time"

```



```

)

// UnifiedModelCatalog is the merged output of all 3 tiers
type UnifiedModelCatalog struct {
    GeneratedAt time.Time           `json:"generated_at"`
    Models      map[string]CatalogEntry `json:"models"`
    Summary     CatalogSummary      `json:"summary"`
}

type CatalogEntry struct {
    ModelID      string `json:"model_id"`
    Provider     string `json:"provider"`
    Name         string `json:"name"`
    SourceTier   string `json:"source_tier" // static,
                provider, dynamic
    Capabilities []string `json:"capabilities"`
    TranslationQuality float64 `json:"translation_quality"`
    LastDiscovered time.Time `json:"last_discovered"`
    VerificationStatus string `json:"verification_status"`
}

type CatalogSummary struct {
    TotalModels int `json:"total_models"`
    StaticModels int `json:"static_models"`
    ProviderModels int `json:"provider_models"`
    DynamicModels int `json:"dynamic_models"`
}

```

6.5 Configuration Examples

```

# configs/verifier.yaml - discovery section
llmsverifier:
  discovery:
    enabled: true
    enable_provider_discovery: true
    enable_dynamic_detection: false
    provider_timeout: 10s
    dynamic_test_count: 3
    cache_ttl: 1h
    providers_to_discover:
      - openai
      - anthropic
      - deepseek
      - groq
      - togetherai

```

6.6 Test Specifications

- **Test File:** internal/verifier/discovery_test.go
- **Test Functions:**
 - TestDiscoveryOrchestrator_StaticTierOnly — verify static models loaded

- TestDiscoveryOrchestrator_AllTiers — mock all tiers, verify merged catalog
- TestProviderDiscovery_DiscoverAll — mock provider clients, verify concurrent discovery
- TestDynamicDetector_DetectCapabilities — mock capability tests
- TestUnifiedModelCatalog_Merge — verify tier 2 + tier 1 merge logic
- **Integration Test:** test/integration/discovery_integration_test.go

6.7 Documentation References

- LLMsVerifier: llm-verifier/providers/model_provider_service.go — provider discovery
 - LLMsVerifier: llm-verifier/capabilities/detector.go — capability detection
-

Chapter 7: API Endpoint Integration

7.1 Section Title & Purpose

Title: "API Integration: New REST Endpoints for Verification, Models, and Scoring" **Purpose:** Document the addition of 3 new REST API endpoint groups (/api/v1/verify, /api/v1/models, /api/v1/score) to HelixTranslate's HTTP API server.

7.2 Specific Content Points

Content Point 7.2.1: Define all new endpoints | Group | Method | Path | Handler Function | Auth Required | |-----|-----|-----|-----|-----| | Verify | POST | /api/v1/verify | HandleVerify | Yes | | Verify | GET | /api/v1/verify/:id/status | HandleVerifyStatus | Yes | | Verify | GET | /api/v1/verify/:id/result | HandleVerifyResult | Yes | | Models | GET | /api/v1/models | HandleListModels | Yes | | Models | GET | /api/v1/models/:id | HandleGetModel | Yes | | Models | POST | /api/v1/models/discover | HandleDiscoverModels | Admin | | Models | GET | /api/v1/models/:id/capabilities | HandleGetCapabilities | Yes | | Score | GET | /api/v1/score | HandleListScores | Yes | | Score | GET | /api/v1/score/:model_id | HandleGetScore | Yes | | Score | POST | /api/v1/score/calculate | HandleCalculateScore | Admin | | Score | POST | /api/v1/score/rank | HandleRankModels | Yes |

Content Point 7.2.2: Modify pkg/api/handler.go

- Add new handler methods to the Handler struct:

```
// pkg/api/handler.go – additions to Handler struct
type Handler struct {
    // ... existing fields ...
    VerifierPipeline *verifier.VerificationPipeline
    ScoringIntegrator *verifier.ScoringIntegrator
}
```

```

    DiscoveryOrchestrator *verifier.DiscoveryOrchestrator
}

```

Content Point 7.2.3: Add route registration

- **File:** pkg/api/handler.go — add to route setup function

```

func (h *Handler) RegisterRoutes(mux *http.ServeMux) {
    // ... existing routes ...

    // LLMsVerifier verification routes
    mux.HandleFunc("/api/v1/verify", h.WithAuth(h.HandleVerify))
    mux.HandleFunc("/api/v1/verify/",
        h.WithAuth(h.HandleVerifySubpath)) // /status, /result

    // LLMsVerifier models routes
    mux.HandleFunc("/api/v1/models",
        h.WithAuth(h.HandleListModels))
    mux.HandleFunc("/api/v1/models/discover",
        h.WithAuth(h.HandleDiscoverModels))
    mux.HandleFunc("/api/v1/models/",
        h.WithAuth(h.HandleModelSubpath))

    // LLMsVerifier scoring routes
    mux.HandleFunc("/api/v1/score",
        h.WithAuth(h.HandleListScores))
    mux.HandleFunc("/api/v1/score/calculate",
        h.WithAuth(h.HandleCalculateScore))
    mux.HandleFunc("/api/v1/score/rank",
        h.WithAuth(h.HandleRankModels))
    mux.HandleFunc("/api/v1/score/",
        h.WithAuth(h.HandleScoreSubpath))
}

```

Content Point 7.2.4: Implement handler functions

- **File to create:** pkg/api/verifier_handlers.go

```

package api

import (
    "encoding/json"
    "net/http"
    "time"

    "digital.vasic.translator/internal/verifier"
)

// VerifyRequest represents a verification request
type VerifyRequest struct {
    ModelID    string    `json:"model_id"`
    Provider   string    `json:"provider,omitempty"`
    Tests      []string    `json:"tests,omitempty"` // Specific
                        tests to run
    Timeout    string    `json:"timeout,omitempty"`
}

```

```

}

// VerifyResponse represents a verification response
type VerifyResponse struct {
    RequestID    string           `json:"request_id"`
    Status       string           `json:"status"`
    ModelID      string           `json:"model_id"`
    SubmittedAt time.Time        `json:"submitted_at"`
}

// HandleVerify initiates a verification run
func (h *Handler) HandleVerify(w http.ResponseWriter, r
    *http.Request) {
    if r.Method != http.MethodPost {
        http.Error(w, "Method not allowed",
            http.StatusMethodNotAllowed)
        return
    }

    var req VerifyRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, "Invalid request body",
            http.StatusBadRequest)
        return
    }

    // Validate request
    if req.ModelID == "" {
        http.Error(w, "model_id is required",
            http.StatusBadRequest)
        return
    }

    // Submit verification asynchronously
    requestID := generateRequestID()
    // ... queue verification job ...

    resp := VerifyResponse{
        RequestID:    requestID,
        Status:       "queued",
        ModelID:      req.ModelID,
        SubmittedAt: time.Now(),
    }

    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusAccepted)
    json.NewEncoder(w).Encode(resp)
}

```

Content Point 7.2.5: Implement models endpoint

```

// ModelsResponse represents the list models response
type ModelsResponse struct {
    Models []ModelInfo `json:"models"`
}

```

```

    Total    int           `json:"total"`
}

type ModelInfo struct {
    ModelID    string `json:"model_id"`
    Name       string `json:"name"`
    Provider   string `json:"provider"`
    Score      float64 `json:"score,omitempty"`
    Capabilities []string `json:"capabilities"`
    Verified   bool    `json:"verified"`
}

// HandleListModels returns all discovered/known models
func (h *Handler) HandleListModels(w http.ResponseWriter, r
    *http.Request) {
    if r.Method != http.MethodGet {
        http.Error(w, "Method not allowed",
            http.StatusMethodNotAllowed)
        return
    }

    // Parse query params
    provider := r.URL.Query().Get("provider")
    minScoreStr := r.URL.Query().Get("min_score")
    verifiedOnly := r.URL.Query().Get("verified_only") == "true"

    // ... fetch and filter models ...

    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(models)
}

```

7.3 Required Tables

Table 7.3.1: API Endpoint Summary

Endpoint	Method	Auth	Request Body	Response
/api/v1/verify	POST	API Key	VerifyRequest	VerifyResponse (202)
/api/v1/verify/:id/status	GET	API Key	—	VerifyStatusResponse
/api/v1/verify/:id/result	GET	API Key	—	PipelineResult
/api/v1/models	GET	API Key	—	ModelsResponse
/api/v1/models/:id	GET	API Key	—	ModelDetailResponse
/api/v1/models/discover	POST	Admin	—	DiscoveryJobResponse (202)
/api/v1/models/:id/capabilities	GET	API Key	—	CapabilitiesResponse
/api/v1/score	GET	API Key	—	ScoresListResponse
/api/v1/score/:model_id	GET	API Key	—	ScoreDetailResponse
/api/v1/score/calculate	POST	Admin	CalculateScoreRequest	ScoreResponse (202)
/api/v1/score/rank	POST	API Key	RankRequest	RankResponse

7.4 Required Code Blocks

Code Block 7.4.1: Complete verifier_handlers.go

```

// pkg/api/verifier_handlers.go
package api

import (
    "encoding/json"
    "net/http"
    "strconv"
    "strings"
    "time"
)

// HandleVerifySubpath routes /verify/{id}/{action} requests
func (h *Handler) HandleVerifySubpath(w http.ResponseWriter, r
    *http.Request) {
    path := strings.TrimPrefix(r.URL.Path, "/api/v1/verify/")
    parts := strings.Split(path, "/")
    if len(parts) < 2 {
        http.Error(w, "Invalid path", http.StatusBadRequest)
        return
    }
    requestID := parts[0]
    action := parts[1]

    switch action {
    case "status":
        h.handleVerifyStatus(w, r, requestID)
    case "result":
        h.handleVerifyResult(w, r, requestID)
    default:
        http.Error(w, "Unknown action", http.StatusNotFound)
    }
}

func (h *Handler) handleVerifyStatus(w http.ResponseWriter, r
    *http.Request, requestID string) {
    // Return verification job status
    status := map[string]any{
        "request_id": requestID,
        "status":     "running", // queued, running, completed,
                        failed
        "progress":  0.5,
    }
    json.NewEncoder(w).Encode(status)
}

// HandleRankModels ranks models for a translation task
func (h *Handler) HandleRankModels(w http.ResponseWriter, r
    *http.Request) {
    if r.Method != http.MethodPost {
        http.Error(w, "Method not allowed",
            http.StatusMethodNotAllowed)
        return
    }
}

```

```

var req struct {
    SourceLang string `json:"source_lang"`
    TargetLang string `json:"target_lang"`
    ProviderFilter []string `json:"provider_filter,omitempty"`
    MinScore float64 `json:"min_score,omitempty"`
}

if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
    http.Error(w, "Invalid request", http.StatusBadRequest)
    return
}

// ... call scoring integrator ...

json.NewEncoder(w).Encode(rankResponse)
}

```

7.5 Configuration Examples

```

# configs/verifier.yaml – API section
llmsverifier:
  api_integration:
    enabled: true
    enable_verify: true
    enable_models: true
    enable_score: true
    rate_limit_per_minute: 60
    require_auth: true

```

7.6 Test Specifications

- **Test File:** pkg/api/verifier_handlers_test.go
- **Test Functions:**
 - TestHandleVerify — POST verification, verify 202 response
 - TestHandleVerify_InvalidBody — malformed request, verify 400
 - TestHandleVerify_MissingModelID — missing model_id, verify 400
 - TestHandleListModels — GET models, verify list response
 - TestHandleListModels_WithFilters — query params filtering
 - TestHandleRankModels — POST rank request, verify ordered results
 - TestHandleRankModels_InvalidLang — invalid language code, verify 400
- **Test File:** test/integration/api_verifier_integration_test.go

7.7 Documentation References

- HelixTranslate: pkg/api/handler.go — existing handler patterns
 - LLMsVerifier: llm-verifier/api/handlers.go — native API handlers
-

Chapter 8: Event System Integration

8.1 Section Title & Purpose

Title: "Event Integration: Subscribing to Translation Events" **Purpose:** Document how LLMsVerifier subscribes to HelixTranslate's event bus for translation lifecycle events, enabling reactive verification and scoring.

8.2 Specific Content Points

Content Point 8.2.1: Define event subscriptions

- **File to modify:** pkg/events/events.go — add new event types
- **File to create:** internal/verifier/event_subscriber.go

Content Point 8.2.2: New event types for LLMsVerifier

```
// pkg/events/events.go – additions
const (
    // ... existing event types ...

    // LLMsVerifier events
    EventTypeVerificationStarted    EventType =
        "verification.started"
    EventTypeVerificationCompleted EventType =
        "verification.completed"
    EventTypeVerificationFailed     EventType =
        "verification.failed"
    EventTypeScoringCompleted       EventType = "scoring.completed"
    EventTypeModelDiscovered        EventType = "model.discovered"
    EventTypeModelRanked            EventType = "model.ranked"
)

// LLMsVerifierEventData holds event-specific data
type LLMsVerifierEventData struct {
    ModelID      string `json:"model_id"`
    Provider     string `json:"provider"`
    Score        float64 `json:"score,omitempty"`
    StepsPassed  int     `json:"steps_passed,omitempty"`
    StepsTotal   int     `json:"steps_total,omitempty"`
    DurationMs   int64   `json:"duration_ms,omitempty"`
}
```

Content Point 8.2.3: Create event subscriber

- **File:** internal/verifier/event_subscriber.go

```
package verifier

import (
    "context"
    "encoding/json"
    "log"
```



```

    "digital.vasic.translator/pkg/events"
)

// EventSubscriber subscribes to translation events and triggers
// verification
type EventSubscriber struct {
    pipeline *VerificationPipeline
    scoring  *ScoringIntegrator
    discovery *DiscoveryOrchestrator
    eventBus *events.EventBus
    config   *EventConfig
}

// EventConfig controls event subscription behavior
type EventConfig struct {
    Enabled bool          `json:"enabled"`
    SubscribeToStarted bool      `json:"subscribe_to_started"`
    SubscribeToCompleted bool      `json:"subscribe_to_completed"`
    SubscribeToErrors bool      `json:"subscribe_to_errors"`
    AutoVerifyOnComplete bool      `json:"auto_verify_on_complete"`
    AutoScoreOnComplete bool      `json:"auto_score_on_complete"`
    PublishResults bool      `json:"publish_results"`
}

// Subscribe registers all event handlers
func (es *EventSubscriber) Subscribe(ctx context.Context) error {
    if !es.config.Enabled {
        log.Println("LLMsVerifier event subscriber disabled")
        return nil
    }

    if es.config.SubscribeToStarted {
        es.eventBus.Subscribe(string(events.EventTypeTranslationStarted),
            es.handleTranslationStarted)
    }
    if es.config.SubscribeToCompleted {
        es.eventBus.Subscribe(string(events.EventTypeTranslationCompleted),
            es.handleTranslationCompleted)
    }
    if es.config.SubscribeToErrors {
        es.eventBus.Subscribe(string(events.EventTypeTranslationError),
            es.handleTranslationError)
    }

    log.Println("LLMsVerifier event subscriber registered")
    return nil
}

// handleTranslationStarted prepares verification for a
// translation session
func (es *EventSubscriber) handleTranslationStarted(event
    events.Event) {
    var data events.TranslationEventData

```

```

    if err := json.Unmarshal(event.Data, &data); err != nil {
        log.Printf("Failed to unmarshal translation started event:
        %v", err)
        return
    }

    log.Printf("Translation started: session=%s, provider=%s,
    model=%s",
    data.SessionID, data.Provider, data.Model)
}

// handleTranslationCompleted triggers post-translation
// verification
func (es *EventSubscriber) handleTranslationCompleted(event
events.Event) {
    var data events.TranslationEventData
    if err := json.Unmarshal(event.Data, &data); err != nil {
        log.Printf("Failed to unmarshal translation completed
        event: %v", err)
        return
    }

    // Auto-verify if enabled
    if es.config.AutoVerifyOnComplete && es.pipeline != nil {
        go func() {
            ctx, cancel :=
            context.WithTimeout(context.Background(),
            es.pipeline.config.TimeoutPerStep*8)
            defer cancel()

            result, err := es.pipeline.RunPipeline(ctx,
            data.Model)
            if err != nil {
                log.Printf("Post-translation verification failed:
                %v", err)
                return
            }

            if es.config.PublishResults {
                es.publishVerificationResult(data.SessionID,
                data.Model, result)
            }
        }()
    }

    // Auto-score if enabled
    if es.config.AutoScoreOnComplete && es.scoring != nil {
        go func() {
            score, err :=
            es.scoring.CalculateScore(context.Background(),
            data.Model)
            if err != nil {

```

```

        log.Printf("Post-translation scoring failed: %v",
err)
        return
    }

    if es.config.PublishResults {
        es.publishScoringResult(data.SessionID,
data.Model, score)
    }
}()
}
}

// publishVerificationResult emits verification result event
func (es *EventSubscriber) publishVerificationResult(sessionID,
modelID string, result *PipelineResult) {
    eventData := events.LLMsVerifierEventData{
        ModelID:    modelID,
        StepsPassed: result.StepsCompleted,
        StepsTotal: result.TotalSteps,
        DurationMs:  result.DurationMs,
    }

    es.eventBus.Publish(events.Event{
        Type:    events.EventTypeVerificationCompleted,
        SessionID: sessionID,
        Data:    mustMarshal(eventData),
    })
}

```

8.3 Required Tables

Table 8.2.1: Event Subscriptions | Subscribe To | Handler | Action | Async |

translation.started	handleTranslationStarted	Log, prepare	No	
translation.completed	handleTranslationCompleted	Trigger verify + score	Yes	
translation.error	handleTranslationError	Log for debugging	No	

Table 8.2.2: Published Events | Event Type | Trigger | Data |

verification.started	Pipeline begins	{model_id, timestamp}
verification.completed	Pipeline succeeds	PipelineResult
verification.failed	Pipeline fails	{model_id, error, steps_passed}
scoring.completed	Score calculation done	{model_id, score}

8.4 Required Code Blocks

Code Block 8.4.1: Event subscriber registration in main

```

// cmd/unified-translator/main.go - additions to main()
func main() {
    // ... existing setup ...

```

```

// Setup LLMsVerifier event subscriber
if cfg.LLMsVerifier.EventIntegration.Enabled {
    eventSubscriber := verifier.NewEventSubscriber(
        verificationPipeline,
        scoringIntegrator,
        discoveryOrchestrator,
        eventBus,
        &cfg.LLMsVerifier.EventIntegration,
    )
    if err := eventSubscriber.Subscribe(ctx); err != nil {
        log.Printf("Failed to subscribe LLMsVerifier events:
        %v", err)
    }
}

// ... rest of main ...
}

```

8.5 Configuration Examples

```

# configs/verifier.yaml – event integration
llmsverifier:
  event_integration:
    enabled: true
    subscribe_to_started: true
    subscribe_to_completed: true
    subscribe_to_errors: true
    auto_verify_on_complete: false
    auto_score_on_complete: true
    publish_results: true

```

8.6 Test Specifications

- **Test File:** internal/verifier/event_subscriber_test.go
- **Test Functions:**
 - TestEventSubscriber_Subscribe — verify all handlers registered
 - TestEventSubscriber_TranslationStarted — verify handler processes event
 - TestEventSubscriber_TranslationCompleted — verify async verify+score triggered
 - TestEventSubscriber_Disabled — verify no-op when disabled
 - TestEventSubscriber_PublishResults — verify result events published

8.7 Documentation References

- HelixTranslate: pkg/events/events.go — event bus interface
 - HelixTranslate: pkg/websocket/hub.go — WebSocket event forwarding
-

PART B: ADVANCED INTEGRATION

Chapter 9: Capabilities Integration (MCPs, LSPs, ACP, Embeddings, RAG)

9.1 Section Title & Purpose

Title: "Capabilities Integration: MCPs (35), LSPs (10), ACP, Embeddings (13), RAG, Skills, Plugins" **Purpose:** Document the integration of LLMsVerifier's capability registry for detecting and exposing 35+ MCPs, 10+ LSPs, ACP, 13+ embeddings providers, RAG, skills, and plugin capabilities.

9.2 Specific Content Points

Content Point 9.2.1: Reference LLMsVerifier capabilities system

- **Source:** `llm-verifier/capabilities/registry.go`, `types.go`, `detector.go`
- **Key types:** `ProviderCapabilities`, `CapabilityMatrix`, `CapabilityQuery`

Content Point 9.2.2: Create `internal/verifier/capabilities.go`

```
package verifier

import (
    "context"

    llmverifier caps "digital.vasic.llmsverifier/llm-verifier/capabilities"
)

// CapabilitiesIntegrator wraps LLMsVerifier capability detection
type CapabilitiesIntegrator struct {
    registry *caps.Registry
    detector *caps.Detector
}

// CapabilitiesReport holds a comprehensive capability report for
// a model
type CapabilitiesReport struct {
    ModelID      string          `json:"model_id"`
    Provider     string          `json:"provider"`
    GeneratedAt  string          `json:"generated_at"`

    // Protocol support
    MCPs []MCPIInfo     `json:"mcps"`
    LSPs []LSPInfo      `json:"lsp"`
}
```

```

    ACP                ACPInfo                `json:"acp"`

    // Embedding providers
    EmbeddingProviders []EmbeddingInfo
    `json:"embedding_providers"`

    // Advanced features
    RAG                RAGInfo                `json:"rag"`
    Skills              []SkillInfo          `json:"skills"`
    Plugins              []PluginInfo        `json:"plugins"`

    // Streaming
    Streaming            StreamingInfo        `json:"streaming"`

    // Network
    Network              NetworkInfo          `json:"network"`

    // Compression
    Compression          CompressionInfo      `json:"compression"`

    // Caching
    Caching              CachingInfo         `json:"caching"`
}

// MCPInfo represents a Model Context Protocol capability
type MCPInfo struct {
    Name          string `json:"name"`
    Supported     bool  `json:"supported"`
    Description   string `json:"description,omitempty"`
}

// LSPInfo represents a Language Server Protocol capability
type LSPInfo struct {
    Name          string `json:"name"`
    Supported     bool  `json:"supported"`
    Description   string `json:"description,omitempty"`
}

// ACPInfo represents Agent Communication Protocol capability
type ACPInfo struct {
    Supported     bool    `json:"supported"`
    Version       string  `json:"version,omitempty"`
    Features      []string `json:"features,omitempty"`
}

// EmbeddingInfo represents an embedding provider capability
type EmbeddingInfo struct {
    Name          string `json:"name"`
    Dimensions     int    `json:"dimensions,omitempty"`
    MaxTokens      int    `json:"max_tokens,omitempty"`
    Supported      bool   `json:"supported"`
}

```

```

// RAGInfo represents Retrieval-Augmented Generation capability
type RAGInfo struct {
    Supported      bool    `json:"supported"`
    VectorDBSupport []string `json:"vector_db_support,omitempty"`
    ChunkingSupport bool    `json:"chunking_support"`
    RerankingSupport bool    `json:"reranking_support"`
}

// SkillInfo represents a model skill
type SkillInfo struct {
    Name          string `json:"name"`
    Proficiency float64 `json:"proficiency" // 0.0 - 1.0`
}

// PluginInfo represents a plugin capability
type PluginInfo struct {
    Name      string `json:"name"`
    Type      string `json:"type"`
    Supported bool   `json:"supported"`
}

```

Content Point 9.2.3: Implement capability query functions

- GetModelCapabilities(ctx, modelID string) (*CapabilitiesReport, error)
- QueryCapabilities(ctx, query *CapabilityQuery) (*CapabilityQueryResult, error)
- GetCapabilityMatrix(ctx) (*CapabilityMatrix, error)
- HasCapability(ctx, modelID, capability string) (bool, error)

Content Point 9.2.4: Create capability API handler

- **File:** pkg/api/capability_handlers.go

```

package api

import (
    "encoding/json"
    "net/http"
)

// HandleGetCapabilities returns full capability report for a
// model
func (h *Handler) HandleGetCapabilities(w http.ResponseWriter, r
    *http.Request) {
    modelID := extractModelID(r)

    report, err :=
        h.capabilities.GetModelCapabilities(r.Context(), modelID)
    if err != nil {
        http.Error(w, err.Error(), http.StatusInternalServerError)
        return
    }
}

```

```

    json.NewEncoder(w).Encode(report)
}

// HandleQueryCapabilities queries capabilities with filters
func (h *Handler) HandleQueryCapabilities(w http.ResponseWriter,
    r *http.Request) {
    var query verifier.CapabilityQuery
    if err := json.NewDecoder(r.Body).Decode(&query); err != nil {
        http.Error(w, "Invalid query", http.StatusBadRequest)
        return
    }

    result, err := h.capabilities.QueryCapabilities(r.Context(),
        &query)
    if err != nil {
        http.Error(w, err.Error(), http.StatusInternalServerError)
        return
    }

    json.NewEncoder(w).Encode(result)
}

```

9.3 Required Tables

Table 9.3.1: MCP Capabilities (35) | Category | MCPs | Count |

----- ----- -----	Filesystem	file-read, file-write, directory-list, file-search	4
Web	web-search, web-browse, web-scrape, api-call	4	
Code	code-execute, code-lint, code-format, code-test, code-review	5	
Database	db-query, db-schema, db-migrate	3	
Git	git-status, git-diff, git-commit, git-branch, git-log	5	
System	shell-exec, process-manage, env-read	3	
Communication	slack-send, email-send, discord-send	3	
Translation	translate-text, detect-language, transliterate	3	
Document	pdf-read, docx-read, epub-read	3	
Other	calculator, datetime, random-generate, image-describe	4	
	Total		35

Table 9.3.2: LSP Capabilities (10) | LSP | Description | |----|-----| | gopls |

Go language server	rust-analyzer	Rust language server	typescript-language-server	TypeScript/JavaScript	pylsp	Python language server	jdtls	Java language server	clangd	C/C++ language server	zls	Zig language server	elixir-ls	Elixir language server	dart-language-server	Dart/Flutter	terraform-ls	Terraform language server
--------------------	---------------	----------------------	----------------------------	-----------------------	-------	------------------------	-------	----------------------	--------	-----------------------	-----	---------------------	-----------	------------------------	----------------------	--------------	--------------	---------------------------

Table 9.3.3: Embedding Providers (13) | Provider | Dimensions | Max Tokens |

----- ----- -----	openai-text-embedding-3-small	1536	8191
openai-text-embedding-3-large	3072	8191	
openai-text-embedding-ada-002	1536	8191	
cohere-embed-english-v3	1024	512	
cohere-embed-multilingual-v3	1024	512	
voyage-3	1024	32000	
voyage-3-large	1024	32000	
voyage-code-3	1024	32000	
mistral-embed	1024	8192	
nommic-embed-text-v1	768	8192	
jina-embeddings-v3	1024	8192	
bge-large-en-v1.5	1024	512	
e5-mistral-7b-instruct	4096	32768	

9.4 Required Code Blocks

Code Block 9.4.1: Capability detection flow


```

// internal/verifier/capabilities.go - detection flow
func (ci *CapabilitiesIntegrator) GetModelCapabilities(ctx
    context.Context, modelID string) (*CapabilitiesReport,
    error) {
    // Query LLMsVerifier capability registry
    caps, err := ci.registry.Query(ctx, &caps.CapabilityQuery{
        Model: modelID,
    })
    if err != nil {
        return nil, fmt.Errorf("failed to query capabilities:
            %w", err)
    }

    report := &CapabilitiesReport{
        ModelID:    modelID,
        Provider:    caps.Provider.Provider,
        GeneratedAt: time.Now().Format(time.RFC3339),
    }

    // Map streaming capabilities
    report.Streaming = StreamingInfo{
        Supported: caps.Provider.Streaming.Supported,
        Types:
            streamingTypesToStrings(caps.Provider.Streaming.Types),
    }

    // Map network capabilities
    report.Network = NetworkInfo{
        HTTPVersions:
            httpVersionsToStrings(caps.Provider.Network.HTTPVersions),
        HTTP2:        caps.Provider.Network.HTTP2Supported,
        HTTP3:        caps.Provider.Network.HTTP3Supported,
    }

    // Map MCPs (35 total)
    report.MCPs = detectMCPs(caps)

    // Map LSPs (10 total)
    report.LSPs = detectLSPs(caps)

    // Map ACP
    report.ACP = ACPInfo{
        Supported: containsProtocol(caps.Provider.Protocols,
            caps.ProtocolACP),
        Version:   "1.0",
    }

    // Map embeddings
    report.EmbeddingProviders = detectEmbeddingProviders(caps)

    // Map RAG
    report.RAG = RAGInfo{
        Supported: caps.Provider.Model_.Embeddings,
    }

```

```

        VectorDBSupport: []string{"qdrant", "weaviate",
            "chromadb"},
        ChunkingSupport: true,
        RerankingSupport: caps.Provider.Model_.Embeddings,
    }

    return report, nil
}

```

9.5 Configuration Examples

```

# configs/verifier.yaml - capabilities
llmsverifier:
  capabilities:
    enabled: true
    auto_detect: true
    cache_ttl: 24h
    detect_mcps: true
    detect_lsps: true
    detect_acp: true
    detect_embeddings: true
    detect_rag: true

```

9.6 Test Specifications

- **Test File:** internal/verifier/capabilities_test.go
- **Test Functions:**
 - TestCapabilitiesIntegrator_GetModelCapabilities — mock registry, verify report
 - TestCapabilitiesIntegrator_QueryCapabilities — filtered query test
 - TestCapabilitiesIntegrator_HasCapability — specific capability check
 - TestMCPDetection — verify all 35 MCPs detected
 - TestLSPDetection — verify all 10 LSPs detected

9.7 Documentation References

- LLMsVerifier: llm-verifier/capabilities/ — full capability system
 - LLMsVerifier: llm-verifier/capabilities/registry.go — provider registry
-

Chapter 10: UX Integration (Display, Filtering, Real-time Status)

10.1 Section Title & Purpose

Title: "UX Integration: Model Scoring Display, Filtering, and Real-time Status"

Purpose: Document the WebSocket and API changes needed to expose LLMsVerifier data to the monitoring dashboard, including model scores, verification status, and real-time filtering.

10.2 Specific Content Points

Content Point 10.2.1: WebSocket event extensions

- **File to modify:** pkg/websocket/hub.go
- Add new event types for LLMsVerifier data:
 - verification.status — real-time verification progress
 - scoring.update — score changes
 - model.discovered — newly discovered models
 - model.ranked — ranking updates

Content Point 10.2.2: Create pkg/websocket/verifier_messages.go

```
package websocket
```

```
import "time"
```

```
// VerificationStatusMessage sent during active verification
```

```
type VerificationStatusMessage struct {  
    Type          string `json:"type"` // "verification.status"  
    SessionID     string `json:"session_id"`  
    ModelID       string `json:"model_id"`  
    StepNumber    int    `json:"step_number"`  
    TotalSteps    int    `json:"total_steps"`  
    StepName      string `json:"step_name"`  
    Status        string `json:"status"` // running, passed, failed  
    Progress      float64 `json:"progress"` // 0.0 - 1.0  
    Timestamp     int64  `json:"timestamp"`  
}
```

```
// ScoreUpdateMessage sent when model scores change
```

```
type ScoreUpdateMessage struct {  
    Type          string `json:"type"` // "scoring.update"  
    ModelID       string `json:"model_id"`  
    ModelName     string `json:"model_name"`  
    OverallScore  float64 `json:"overall_score"`  
    Components   map[string]float64 `json:"components"`  
    Rank          int    `json:"rank"`  
    PreviousRank int    `json:"previous_rank"`  
    Timestamp     int64  `json:"timestamp"`  
}
```

```
// ModelFilterRequest from client to filter models
type ModelFilterRequest struct {
    Type          string    `json:"type"` // "models.filter"
    Providers      []string  `json:"providers,omitempty"`
    MinScore       float64   `json:"min_score,omitempty"`
    MaxScore       float64   `json:"max_score,omitempty"`
    Capabilities   []string  `json:"capabilities,omitempty"`
    VerifiedOnly   bool      `json:"verified_only,omitempty"`
    SortBy         string    `json:"sort_by"` // score, name,
                    provider, recency
    SortDirection string    `json:"sort_direction"` // asc, desc
}
```

```
// ModelsListResponse filtered model list
type ModelsListResponse struct {
    Type          string    `json:"type"` // "models.list"
    Models        []ModelEntry `json:"models"`
    Total         int       `json:"total"`
    Filtered      int       `json:"filtered"`
    Timestamp     int64     `json:"timestamp"`
}
```

```
type ModelEntry struct {
    ModelID       string    `json:"model_id"`
    Name          string    `json:"name"`
    Provider      string    `json:"provider"`
    Score         float64   `json:"score"`
    ScoreSuffix   string    `json:"score_suffix"`
    Capabilities  []string  `json:"capabilities"`
    Verified      bool      `json:"verified"`
    Status        string    `json:"status"`
}
```

Content Point 10.2.3: Real-time status broadcasting

- **File to create:** pkg/websocket/verifier_broadcast.go

```
package websocket
```

```
import (
    "encoding/json"
    "log"
    "time"
)
```

```
// VerifierBroadcaster handles LLMsVerifier WebSocket broadcasts
type VerifierBroadcaster struct {
    hub *Hub
}
```

```
// BroadcastVerificationStatus sends verification progress to all
    clients
```

```

func (vb *VerifierBroadcaster)
    BroadcastVerificationStatus(sessionID, modelID string,
        stepNum, totalSteps int, stepName, status string,
        progress float64) {
    msg := VerificationStatusMessage{
        Type:         "verification.status",
        SessionID:    sessionID,
        ModelID:      modelID,
        StepNumber:   stepNum,
        TotalSteps:   totalSteps,
        StepName:     stepName,
        Status:       status,
        Progress:     progress,
        Timestamp:    time.Now().Unix(),
    }

    data, _ := json.Marshal(msg)
    vb.hub.Broadcast(data)
}

// BroadcastScoreUpdate sends score changes
func (vb *VerifierBroadcaster) BroadcastScoreUpdate(modelID,
    modelName string, score float64, components
    map[string]float64, rank, prevRank int) {
    msg := ScoreUpdateMessage{
        Type:         "scoring.update",
        ModelID:      modelID,
        ModelName:    modelName,
        OverallScore: score,
        Components:   components,
        Rank:         rank,
        PreviousRank: prevRank,
        Timestamp:    time.Now().Unix(),
    }

    data, _ := json.Marshal(msg)
    vb.hub.Broadcast(data)
}

```

10.3 Required Tables

Table 10.3.1: WebSocket Message Types

Type	Direction	Purpose	Payload
verification.status	Server->Client	Progress updates	VerificationStatusMessage
scoring.update	Server->Client	Score changes	ScoreUpdateMessage
model.discovered	Server->Client	New model found	{model_id, provider}
models.filter	Client->Server	Request filtered list	ModelFilterRequest
models.list	Server->Client	Filtered model list	ModelsListResponse

Table 10.3.2: Model Filtering Options

Filter	Type	Example
Provider	string[]	["openai", "anthropic"]
Min/Max Score	float	min_score: 7.0
Capabilities	string[]	["streaming", "function_calling"]
Verified Only	bool	true
Sort By	enum	score, name, provider, recency

10.4 Required Code Blocks

Code Block 10.4.1: Filter handler for WebSocket

```
// pkg/api/handler.go – WebSocket filter handler addition
func (h *Handler) handleModelFilter(w http.ResponseWriter, r
    *http.Request) {
    var req websocket.ModelFilterRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, "Invalid filter", http.StatusBadRequest)
        return
    }

    // Get all models
    models, _ := h.discovery.GetCatalog(r.Context())

    // Apply filters
    filtered := applyModelFilters(models, &req)

    // Sort
    sortModels(filtered, req.SortBy, req.SortDirection)

    resp := websocket.ModelsListResponse{
        Type:      "models.list",
        Models:    filtered,
        Total:      len(models),
        Filtered:   len(filtered),
        Timestamp: time.Now().Unix(),
    }

    json.NewEncoder(w).Encode(resp)
}
```

10.5 Configuration Examples

```
# WebSocket configuration for LLMsVerifier events
llmsverifier:
  ux:
    enable_realtime_updates: true
    score_update_interval: 60s
    verification_progress_updates: true
    default_sort: score
    default_sort_direction: desc
    page_size: 25
```

10.6 Test Specifications

- **Test File:** pkg/websocket/verifier_messages_test.go
 - **Test Functions:**
 - TestVerificationStatusMessage_Marshal — verify JSON serialization
 - TestModelFilterRequest_Validate — verify filter validation
 - TestApplyModelFilters — unit test filter logic
 - TestSortModels — verify sorting by different fields
-

PART C: QUALITY ASSURANCE

Chapter 11: Testing Strategy (100% Coverage, 17 Challenges, Anti-Bluff)

11.1 Section Title & Purpose

Title: "Testing Strategy: 100% Coverage, 17 Challenge Scripts, and Anti-Bluff Framework" **Purpose:** Document the comprehensive testing strategy including unit tests, integration tests, 17 challenge verification scripts, and the anti-bluff detection framework.

11.2 Specific Content Points

Content Point 11.2.1: Test file inventory | Category | Count | Location | Pattern |
|-----|-----|-----|-----| | Unit tests | 50+ | internal/verifier/
*_test.go | TestFunctionName_Scenario | | Integration tests | 15+ | test/
integration/verifier*_test.go | //go:build integration | | Challenge
scripts | 17 | test/challenges/ | Challenge definitions | | API tests | 10+ | pkg/
api/verifier*_test.go | HTTP handler tests | | Mock implementations | 5+ |
test/mocks/verifier*_test.go | Mock clients | | E2E tests | 3+ | test/e2e/
verifier*_test.go | //go:build e2e |

Content Point 11.2.2: Unit test specifications for each module

For internal/verifier/pipeline_test.go:

```
// Test functions:
func TestVerificationPipeline_AllStepsPass(t
    *testing.T) // Happy path
func TestVerificationPipeline_CriticalStepFails(t
    *testing.T) // Critical failure
func TestVerificationPipeline_NonCriticalStepFails(t
    *testing.T) // Non-critical failure
func TestVerificationPipeline_Timeout(t
    *testing.T) // Step timeout
```

```

func TestVerificationPipeline_ContextCancel(t
    *testing.T)           // Context cancellation
func TestVerificationPipeline_SkipSteps(t
    *testing.T)           // Skip configuration
func TestVerificationPipeline_InvalidModel(t
    *testing.T)           // Invalid model ID

```

For internal/verifier/scoring_test.go:

```

func TestScoringIntegrator_CalculateScore(t *testing.T)
func TestScoringIntegrator_CalculateScore_InvalidModel(t
    *testing.T)
func TestScoringIntegrator_RankModels(t *testing.T)
func TestScoringIntegrator_IsModelAcceptable(t *testing.T)
func TestScoringIntegrator_WeightsSum(t
    *testing.T)           // Verify weights sum to 1.0
func TestScoreCache_GetHit(t *testing.T)
func TestScoreCache_GetMiss(t *testing.T)
func TestScoreCache_Expiration(t *testing.T)

```

For internal/providers/llmsverifier/adapter_test.go:

```

func TestLLMsVerifierAdapter_Translate(t *testing.T)
func TestLLMsVerifierAdapter_Translate_Error(t *testing.T)
func TestLLMsVerifierAdapter_GetProviderName(t *testing.T)
func TestLLMsVerifierAdapter_ProviderSwitch(t
    *testing.T)           // Table-driven, 30+ providers
func TestNewLLMsVerifierAdapter_InvalidConfig(t *testing.T)
func TestNewLLMsVerifierAdapter_EmptyAPIKey(t *testing.T)

```

Content Point 11.2.3: 17 Challenge scripts Based on LLMsVerifier's challenge system:

#	Challenge	Purpose	Test File	1
01	challenge_basic_translation	Basic EN->SR translation	test/challenges/01_basic_translation_test.go	1
02	challenge_long_text	10K+ token text handling	test/challenges/02_long_text_test.go	2
03	challenge_cyrillic_script	Serbian Cyrillic accuracy	test/challenges/03_cyrillic_script_test.go	3
04	challenge_latin_script	Serbian Latin accuracy	test/challenges/04_latin_script_test.go	4
05	challenge_technical_terms	Technical terminology	test/challenges/05_technical_terms_test.go	5
06	challenge_cultural_context	Cultural adaptation	test/challenges/06_cultural_context_test.go	6
07	challenge_multilingual	Multi-language support	test/challenges/07_multilingual_test.go	7
08	challenge_format_preservation	FB2/EPUB format preservation	test/challenges/08_format_preservation_test.go	8
09	challenge_streaming_response	Streaming API handling	test/challenges/09_streaming_response_test.go	9
10	challenge_error_recovery	Error handling and retry	test/challenges/10_error_recovery_test.go	10
11	challenge_rate_limiting	Rate limit compliance	test/challenges/11_rate_limiting_test.go	11
12	challenge_context_window	Max context utilization	test/challenges/12_context_window_test.go	12
13	challenge_code_in_text	Code snippet preservation	test/challenges/13_code_in_text_test.go	13
14	challenge_html_entities	HTML entity handling	test/challenges/14_html_entities_test.go	14
15	challenge_poetry_meter	Poetry/rhythmic text	test/challenges/15_poetry_meter_test.go	15
16	challenge_idioms			16

Idiomatic expression translation | test/challenges/16_idioms_test.go || 17 |
challenge_anti_bluff | Hallucination detection | test/challenges/
17_anti_bluff_test.go |

Content Point 11.2.4: Anti-bluff framework

- File: test/challenges/17_anti_bluff_test.go

```
package challenges
```

```
import (  
    "strings"  
    "testing"
```

```
  
    "github.com/stretchr/testify/assert"  
)
```

```
// AntiBluffTest verifies that model outputs are faithful to input  
// and don't hallucinate or invent content
```

```
type AntiBluffTest struct {  
    InputText          string  
    ExpectedPatterns []string // Must appear in output  
    ForbiddenPatterns []string // Must NOT appear in output  
    MaxLengthDelta    float64 // Max ratio of output/input  
                                length  
}
```

```
var antiBluffTests = []AntiBluffTest{  
    {  
        InputText:          "The cat sat on the mat.",  
        ExpectedPatterns: []string{"cat", "mat"},  
        ForbiddenPatterns: []string{"dog", "impossible",  
                                "magical"},  
        MaxLengthDelta:    3.0, // Translation can be up to 3x  
                                original  
    },  
    {  
        InputText:          "function add(a, b) { return a + b; }",  
        ExpectedPatterns: []string{"add", "a", "b", "return"},  
        ForbiddenPatterns: []string{"magic", "random",  
                                "undefined"},  
        MaxLengthDelta:    2.0,  
    },  
    // ... more anti-bluff tests  
}
```

```
func TestAntiBluff(t *testing.T) {  
    for _, test := range antiBluffTests {  
        t.Run(test.InputText[:min(30, len(test.InputText))],  
            func(t *testing.T) {  
                // Run translation through each provider  
                for _, provider := range getTestProviders() {  
                    output := translateWithProvider(t, provider,  
                        test.InputText)
```

```

        // Check expected patterns present
        for _, pattern := range test.ExpectedPatterns {
            assert.Contains(t, output, pattern,
                "Provider %s: expected pattern %q not
found", provider, pattern)
        }

        // Check forbidden patterns absent
        for _, pattern := range test.ForbiddenPatterns {
            assert.NotContains(t, output, pattern,
                "Provider %s: forbidden pattern %q
found", provider, pattern)
        }

        // Check length ratio
        ratio := float64(len(output)) /
float64(len(test.InputText))
        assert.LessOrEqual(t, ratio, test.MaxLengthDelta,
            "Provider %s: output too long (ratio %.2f)",
            provider, ratio)
    }
}
}
}
}

```

Content Point 11.2.5: Coverage targets | Package | Current Coverage | Target | Gap |

-----	-----	-----	-----	internal/verifier/	0% 100% +100%
-----	-----	-----	-----	internal/providers/llmsverifier/	0% 100% +100%
-----	-----	-----	-----	internal/services/	N/A 100% New
-----	-----	-----	-----	pkg/api/	32.8% 80% +47.2%
-----	-----	-----	-----	pkg/verification/	N/A 90% New
-----	-----	-----	-----	Overall	43.6% 100% +56.4%

11.3 Required Tables

Table 11.3.1: Test File Inventory | File | Package | Tests | Type |

-----	-----	-----	-----	internal/verifier/pipeline_test.go	verifier 7
-----	-----	-----	-----	Unit internal/verifier/scoring_test.go	verifier 8 Unit
-----	-----	-----	-----	internal/verifier/discovery_test.go	verifier 5 Unit
-----	-----	-----	-----	internal/verifier/event_subscriber_test.go	verifier 5 Unit
-----	-----	-----	-----	internal/verifier/capabilities_test.go	verifier 4 Unit
-----	-----	-----	-----	internal/providers/llmsverifier/adapter_test.go	llmsverifier 6 Unit
-----	-----	-----	-----	internal/providers/llmsverifier/config_test.go	llmsverifier 4 Unit
-----	-----	-----	-----	pkg/api/verifier_handlers_test.go	api 7 Unit
-----	-----	-----	-----	pkg/websocket/verifier_messages_test.go	websocket 4 Unit
-----	-----	-----	-----	test/integration/verification_pipeline_test.go	integration 3 Integration
-----	-----	-----	-----	test/integration/scoring_integration_test.go	integration 3 Integration
-----	-----	-----	-----	test/integration/discovery_integration_test.go	integration 2 Integration
-----	-----	-----	-----	test/integration/api_verifier_integration_test.go	integration 4 Integration
-----	-----	-----	-----	test/e2e/verifier_end_to_end_test.go	e2e 2 E2E
-----	-----	-----	-----	test/challenges/	challenges 17 Challenge

11.4 Required Code Blocks

Code Block 11.4.1: Mock LLMsVerifier client for testing

```
// test/mocks/verifier_client_mock.go
package mocks

import (
    "context"
    "sync"

    llmverifier "digital.vasic.llmsverifier/llm-verifier/sdk/go"
)

// MockLLMsVerifierClient implements the LLMsVerifier SDK client
// interface for testing
type MockLLMsVerifierClient struct {
    mu sync.Mutex

    Responses map[string]string
    Errors     map[string]error
    CallCount  int
    LastRequest *llmverifier.CompletionRequest
}

func NewMockLLMsVerifierClient() *MockLLMsVerifierClient {
    return &MockLLMsVerifierClient{
        Responses: make(map[string]string),
        Errors:     make(map[string]error),
    }
}

func (m *MockLLMsVerifierClient) CreateCompletion(ctx
    context.Context, req llmverifier.CompletionRequest)
    (*llmverifier.CompletionResponse, error) {
    m.mu.Lock()
    defer m.mu.Unlock()

    m.CallCount++
    m.LastRequest = &req

    if err, ok := m.Errors[req.Model]; ok {
        return nil, err
    }

    resp := &llmverifier.CompletionResponse{
        Text: m.Responses[req.Model],
    }
    if resp.Text == "" {
        resp.Text = "Mock translation response"
    }
}
```

```

    return resp, nil
}

```

Code Block 11.4.2: Test helper for provider adapter

```

// test/utils/verifier_test_helpers.go
package utils

import (
    "testing"

    "digital.vasic.translator/internal/providers/llmsverifier"
    "digital.vasic.translator/test/mocks"
)

// NewTestAdapter creates an LLMsVerifier adapter with a mock
// client
func NewTestAdapter(t *testing.T, providerName, modelName string)
    *llmsverifier.LLMsVerifierAdapter {
    mockClient := mocks.NewMockLLMsVerifierClient()
    mockClient.Responses[modelName] = "Test translation output"

    adapter, err :=
        llmsverifier.NewLLMsVerifierAdapter(llm.ProviderConfig{
            Provider: providerName,
            Model:    modelName,
            APIKey:   "test-key",
        })
    if err != nil {
        t.Fatalf("Failed to create test adapter: %v", err)
    }

    return adapter
}

```

11.5 Configuration Examples

```

# test/testdata/verifier_test_config.yaml
llmsverifier:
  enabled: true
  verification:
    enabled: true
    timeout_per_step: 5s
    max_concurrent: 2
  scoring:
    enabled: true
    min_acceptable_score: 5.0
  weights:
    response_speed: 0.25
    model_efficiency: 0.20
    cost_effectiveness: 0.25
    capability: 0.20
    recency: 0.10

```

```
providers:
  - name: "mock"
    endpoint: "http://localhost:9999"
    api_key: "test-key"
    model: "mock-model"
```

11.6 Test Specifications (Detailed)

Integration Test Spec: test/integration/verification_pipeline_test.go

```
//go:build integration
```

```
package integration
```

```
import (
    "context"
    "testing"
    "time"

    "digital.vasic.translator/internal/verifier"
)

func TestVerificationPipeline_Integration(t *testing.T) {
    ctx := context.Background()

    // Load test config
    cfg := loadTestConfig(t, "testdata/verifier_test_config.yaml")

    // Create pipeline
    pipeline := verifier.NewVerificationPipeline(cfg.Verification)

    // Run against real provider (requires API key)
    result, err := pipeline.RunPipeline(ctx, "gpt-4")

    assert.NoError(t, err)
    assert.NotNil(t, result)
    assert.Equal(t, 8, result.TotalSteps)
    assert.True(t, result.StepsCompleted >= 4, "At least half the
        steps should pass")
}
```

11.7 Documentation References

- LLMsVerifier: [llm-verifier/challenges/](#) — challenge framework
 - HelixTranslate: [test/](#) — existing test structure
-

Chapter 12: Documentation (7 Primary + 7 Governance)

12.1 Section Title & Purpose

Title: "Documentation: 7 Primary Documents and 7 Governance Documents"
Purpose: Document the 14 documentation files needed for the integration, covering both main repo and submodule guidance.

12.2 Specific Content Points

Content Point 12.2.1: 7 Primary Documentation Files

#	Document	Path	Purpose	Key Sections
1	Integration Guide	Documentation/LLMSVERIFIER_INTEGRATION_GUIDE.md	Master integration document	Architecture, setup, configuration
2	API Reference	Documentation/LLMSVERIFIER_API_REFERENCE.md	New endpoint documentation	All 11 endpoints, request/response schemas
3	Provider Reference	Documentation/LLMSVERIFIER_PROVIDERS.md	30+ provider matrix	Provider names, models, env vars, capabilities
4	Configuration Guide	Documentation/LLMSVERIFIER_CONFIGURATION.md	Config options reference	All 16+ env vars, YAML structure, defaults
5	Testing Guide	Documentation/LLMSVERIFIER_TESTING.md	Test writing guidance	Unit, integration, challenge patterns
6	Scoring Guide	Documentation/LLMSVERIFIER_SCORING.md	Scoring system docs	5 components, weights, interpretation
7	Troubleshooting	Documentation/LLMSVERIFIER_TROUBLESHOOTING.md	Common issues	Error codes, debug steps, FAQ

Content Point 12.2.2: 7 Governance Documentation Files

| # | Document | Path | Purpose | Audience | |-----|-----|-----|-----| | 1 | **CLAUDE.md Update** | CLAUDE.md (modify) | Claude Code guidance for integration | Claude Code | | 2 | **AGENTS.md Update** | AGENTS.md (modify) | AI

agent project reference | AI agents || 3 | **CONTRIBUTING** |
CONTRIBUTING_LLMVERIFIER.md | Contribution guidelines | Contributors || 4 |
Architecture Decision Record | Documentation/ADR-001-LLMsVerifier-
Integration.md | Design decisions | Technical leads || 5 | **Security Guide** |
Documentation/LLMSVERIFIER_SECURITY.md | API key management, RBAC |
Security team || 6 | **Operations Runbook** | Documentation/
LLMSVERIFIER_OPERATIONS.md | Deployment, monitoring | DevOps || 7 |
Submodule Guide | LLMsVerifier/INTEGRATION_WITH_HELIXTRANSLATE.md |
LLMsVerifier-side docs | LLMsVerifier maintainers |

Content Point 12.2.3: CLAUDE.md modifications

- Add section: "## LLMsVerifier Integration"
- Document:
 - Module path: digital.vasic.llmsverifier
 - Key packages: internal/verifier/, internal/providers/llmsverifier/
 - Configuration file: configs/verifier.yaml
 - Environment variables: all 16+ LLMSVERIFIER_* vars
 - API endpoints: all 11 new endpoints
 - Testing: how to run verifier-specific tests
 - Definition of Done: all verification tests pass

Content Point 12.2.4: AGENTS.md modifications

- Add section: "### LLMsVerifier Module"
- Document:
 - What LLMsVerifier provides (30+ providers, verification, scoring)
 - How to add a new provider via LLMsVerifier
 - How to run verification pipeline
 - How to interpret scores
 - File structure additions

12.3 Required Tables

Table 12.3.1: Documentation File Matrix

Document	Type	New/Modify	Lines
Owner ----- ----- ----- ----- ----- INTEGRATION_GUIDE	Markdown	New	300
Core dev API_REFERENCE	Markdown	New	400
API dev PROVIDERS	Markdown	New	200
Provider dev CONFIGURATION	Markdown	New	250
Config dev TESTING	Markdown	New	200
QA lead SCORING	Markdown	New	150
Scoring dev TROUBLESHOOTING	Markdown	New	200
Support CLAUDE.md	Markdown	Modify	+100
Core dev AGENTS.md	Markdown	Modify	+100
Core dev CONTRIBUTING	Markdown	New	100
Core dev ADR-001	Markdown	New	150
Architect SECURITY	Markdown	New	150
Security OPERATIONS	Markdown	New	200
DevOps INTEGRATION_WITH_HELIXTRANSLATE	Markdown	New	200
LLMsVerifier dev			

Table 12.3.2: CLAUDE.md Additions (Section Outline)

- ## LLMsVerifier Integration
 - Overview
 - Module Setup

- go.mod changes
- Environment variables (16+)
- Configuration
 - configs/verifier.yaml
 - Default values
- Key Packages
 - internal/verifier/
 - internal/providers/llmsverifier/
- API Endpoints
 - /api/v1/verify
 - /api/v1/models
 - /api/v1/score
- Running Tests
 - Unit: go test ./internal/verifier/...
 - Integration: go test -tags=integration ./test/integration/...
 - Challenges: go test -tags=challenge ./test/challenges/...
- Definition of Done
 - All 8 verification steps pass
 - Score >= 7.0 for primary models
 - 100% test coverage on new code

12.4 Required Code Blocks

Code Block 12.4.1: INTEGRATION_GUIDE.md outline

```
# LLMsVerifier Integration Guide

## 1. Overview
HelixTranslate integrates LLMsVerifier to expand from 9 to 30+ LLM
    providers,
add 8-step model verification, 5-component scoring, and 3-tier
    model discovery.

## 2. Architecture
[Diagram placeholder: HelixTranslate <--> LLMsVerifier adapter <-->
    30+ providers]

## 3. Setup
### 3.1 Clone LLMsVerifier
git submodule add git@github.com:vasic-digital/LLMsVerifier.git

### 3.2 Update go.mod
See go.mod changes for require/replace directives.

### 3.3 Configure Environment
Set all 16+ LLMSVERIFIER_* environment variables.

### 3.4 Configure Providers
Edit configs/verifier.yaml to add provider API keys.

## 4. Configuration Reference
[Full table of all config options]
```



```
## 5. API Usage
[Examples of all 11 new endpoints]

## 6. Verification Pipeline
[Description of 8-step pipeline]

## 7. Scoring System
[Description of 5-component scoring]

## 8. Troubleshooting
[Common issues and solutions]
```

Code Block 12.4.2: AGENTS.md additions

```
### LLMsVerifier Module

LLMsVerifier (digital.vasic.llmsverifier) provides:
- **30+ LLM providers** via adapter pattern
- **8-step verification pipeline** at translation startup
- **5-component scoring** for model ranking
- **3-tier model discovery** (static, provider, dynamic)
- **Capability detection**: 35 MCPs, 10 LSPs, ACP, 13 embeddings,
  RAG

**Key files:**
- `internal/verifier/` – verification, scoring, discovery
- `internal/providers/llmsverifier/` – adapter implementation
- `configs/verifier.yaml` – configuration

**Adding a new provider:**
1. Add provider config to `configs/verifier.yaml`
2. Set API key environment variable
3. Provider is automatically available via adapter

**Running verification:**

curl -X POST http://localhost:8080/api/v1/verify
-H "Authorization: Bearer $API_KEY"
-d '{"model_id": "gpt-4"}'
```

12.5 Configuration Examples

- `configs/verifier.yaml` (complete example in Chapter 2)
- `.env.example` (all 16+ env vars)
- `config.json` snippet for LLMsVerifier section

12.6 Test Specifications

N/A (this chapter is about documentation itself)

12.7 Documentation References

- HelixTranslate: CLAUDE.md — existing structure
 - HelixTranslate: AGENTS.md — existing structure
 - LLMsVerifier: README.md — upstream documentation
-

APPENDIX: IMPLEMENTATION CHECKLIST

A. File Creation Checklist

New Files (22+)

- ☐ internal/verifier/pipeline.go
- ☐ internal/verifier/steps.go
- ☐ internal/verifier/orchestrator.go
- ☐ internal/verifier/startup_hook.go
- ☐ internal/verifier/scoring.go
- ☐ internal/verifier/scoring_cache.go
- ☐ internal/verifier/discovery.go
- ☐ internal/verifier/discovery_static.go
- ☐ internal/verifier/discovery_provider.go
- ☐ internal/verifier/discovery_dynamic.go
- ☐ internal/verifier/event_subscriber.go
- ☐ internal/verifier/capabilities.go
- ☐ internal/verifier/config.go
- ☐ internal/providers/llmsverifier/adaptor.go
- ☐ internal/providers/llmsverifier/config.go

- ☐ internal/providers/llmsverifier/provider.go
- ☐ internal/services/llmsverifier_score_adapter.go
- ☐ pkg/api/verifier_handlers.go
- ☐ pkg/api/capability_handlers.go
- ☐ pkg/websocket/verifier_messages.go
- ☐ pkg/websocket/verifier_broadcast.go
- ☐ configs/verifier.yaml

Modified Files (8+)

- ☐ go.mod (add require + replace)
- ☐ internal/config/config.go (add LLMsVerifierConfig)
- ☐ pkg/translator/llm/llm.go (add ProviderLLMsVerifier)
- ☐ pkg/api/handler.go (add routes + handlers)
- ☐ pkg/events/events.go (add event types)
- ☐ cmd/unified-translator/main.go (add startup hook)
- ☐ CLAUDE.md (add LLMsVerifier section)
- ☐ AGENTS.md (add LLMsVerifier section)

Test Files (20+)

- ☐ internal/verifier/*_test.go (7 files)
- ☐ internal/providers/llmsverifier/*_test.go (3 files)
- ☐ pkg/api/verifier*_test.go (2 files)
- ☐ pkg/websocket/verifier*_test.go (1 file)
- ☐ test/integration/verifier*_test.go (4 files)
- ☐

- ☐ test/e2e/verifier*_test.go (1 file)
- ☐ test/challenges/*_test.go (17 files)
- ☐ test/mocks/verifier*.go (2 files)

B. Configuration Checklist

Environment Variables (16+)

- ☐ LLMSVERIFIER_ENABLED
- ☐ LLMSVERIFIER_BASE_URL
- ☐ LLMSVERIFIER_API_KEY
- ☐ LLMSVERIFIER_DB_PATH
- ☐ LLMSVERIFIER_VERIFICATION_ENABLED
- ☐ LLMSVERIFIER_SCORING_ENABLED
- ☐ LLMSVERIFIER_DISCOVERY_ENABLED
- ☐ LLMSVERIFIER_AUTO_VERIFY
- ☐ LLMSVERIFIER_AUTO_SCORE
- ☐ LLMSVERIFIER_MAX_CONCURRENT
- ☐ LLMSVERIFIER_TIMEOUT
- ☐ LLMSVERIFIER_WEIGHT_SPEED
- ☐ LLMSVERIFIER_WEIGHT EFFICIENCY
- ☐ LLMSVERIFIER_WEIGHT_COST
- ☐ LLMSVERIFIER_WEIGHT CAPABILITY
- ☐ LLMSVERIFIER_WEIGHT RECENCY

Provider API Keys (21 new)

☐

- ☐ COHERE_API_KEY
- ☐ GROQ_API_KEY
- ☐ XAI_API_KEY
- ☐ TOGETHERAI_API_KEY
- ☐ REPLICATE_API_TOKEN
- ☐ CLOUDFLARE_API_KEY
- ☐ HYPERBOLIC_API_KEY
- ☐ SAMBANOVA_API_KEY
- ☐ SILICONFLOW_API_KEY
- ☐ UPSTAGE_API_KEY
- ☐ PUBLICAI_API_KEY
- ☐ KIMI_API_KEY
- ☐ KIMICODE_API_KEY
- ☐ CEREBRAS_API_KEY
- ☐ MODAL_API_KEY
- ☐ NIA_API_KEY
- ☐ MISTRAL_API_KEY
- ☐ VULAVULA_API_KEY
- ☐ NOVITA_API_KEY
- ☐ SARVAM_API_KEY
- ☐ KILO_API_KEY

C. API Endpoint Checklist (11 endpoints)

- ☐
- ☐ POST /api/v1/verify
- ☐

- ☐ GET /api/v1/verify/:id/status
- ☐ GET /api/v1/verify/:id/result
- ☐ GET /api/v1/models
- ☐ GET /api/v1/models/:id
- ☐ POST /api/v1/models/discover
- ☐ GET /api/v1/models/:id/capabilities
- ☐ GET /api/v1/score
- ☐ GET /api/v1/score/:model_id
- ☐ POST /api/v1/score/calculate
- ☐ POST /api/v1/score/rank

D. Verification Pipeline Steps (8 steps)

- ☐ Step 1: Model Existence Check
- ☐ Step 2: API Connectivity Test
- ☐ Step 3: Authentication Validation
- ☐ Step 4: Basic Completion Test
- ☐ Step 5: Capability Detection
- ☐ Step 6: Translation Quality Test
- ☐ Step 7: Latency Benchmark
- ☐ Step 8: Error Handling Verification

E. Scoring Components (5 components)

- ☐ Response Speed (weight: 0.25)
- ☐ Model Efficiency (weight: 0.20)
- ☐

- ☐ Cost Effectiveness (weight: 0.25)
- ☐ Capability (weight: 0.20)
- ☐ Recency (weight: 0.10)

F. Documentation Checklist (14 documents)

- ☐ Documentation/LLMSVERIFIER_INTEGRATION_GUIDE.md
- ☐ Documentation/LLMSVERIFIER_API_REFERENCE.md
- ☐ Documentation/LLMSVERIFIER_PROVIDERS.md
- ☐ Documentation/LLMSVERIFIER_CONFIGURATION.md
- ☐ Documentation/LLMSVERIFIER_TESTING.md
- ☐ Documentation/LLMSVERIFIER_SCORING.md
- ☐ Documentation/LLMSVERIFIER_TROUBLESHOOTING.md
- ☐ CLAUDE.md (updated)
- ☐ AGENTS.md (updated)
- ☐ CONTRIBUTING_LLMVERIFIER.md
- ☐ ADR-001-LLMsVerifier-Integration.md
- ☐ Documentation/LLMSVERIFIER_SECURITY.md
- ☐ Documentation/LLMSVERIFIER_OPERATIONS.md
- ☐ LLMsVerifier/INTEGRATION_WITH_HELIXTRANSLATE.md

End of Content Specifications Document